ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
UNIVERSITY OF PIRAEUS

ΕΞΟΡΥΞΗ ΓΝΩΣΗΣ ΑΠΟ ΜΟΥΣΙΚΑ ΔΕΔΟΜΕΝΑ

Συγκριτική Ανάλυση Αλγορίθμων Κατηγοριοποίησης και Συσταδοποίησης στο περιβάλλον WEKA

Καθηγητής: Μ. Φιλιππάκης Φεβρουάριος 2026

Θεόδωρος Τσούτσουρας (me25040) & Γιώργος Κολοβός (me25019)

Ιανουάριος - Φεβρουάριος 2026

Μετατροπή dataset για συμβατότητα με WEKA

Πρόβλημα που αντιμετωπίστηκε (WEKA CSV parsing)

Κατά την εισαγωγή του αρχικού αρχείου music_genre.csv στο WEKA εμφανίστηκαν σφάλματα τύπου CSV parsing, όπως:

- wrong number of values. Read 4, expected 18 (συνήθως στη γραμμή 2)
- Token[EOL] / πρόωρο τέλος γραμμής (end-of-line)
- Problem encountered on line: 210 με μήνυμα null

Το βασικό αίτιο ήταν ότι το WEKA CSVLoader είναι ιδιαίτερα αυστηρό στην ερμηνεία εισαγωγικών/οριοθέτησης πεδίων. Στο dataset υπήρχαν:

- **αποστρόφοι** μέσα σε text τιμές (π.χ. *Royksopp's*, *Doin' It Right*), που σε ορισμένες ρυθμίσεις του CSVLoader μπορεί να εκληφθούν λανθασμένα ως χαρακτήρας περίφραξης (enclosure), προκαλώντας "ανοικτό quote" και αποτυχία parsing στη γραμμή.
- περιπτώσεις με **διπλά εισαγωγικά** μέσα σε τίτλους/κειμενικά πεδία, που οδηγούσαν σε parsing exceptions (π.χ. "null error" σε συγκεκριμένες γραμμές).

Για την επίλυση του προβλήματος δοκιμάστηκαν διαδοχικές προσεγγίσεις μετασχηματισμού, με τελικό στόχο την παραγωγή ενός αρχείου που να ανοίγει αξιόπιστα στο WEKA.

Προσέγγιση 1: Επανεξαγωγή CSV με pandas και QUOTE_NONNUMERIC (Version 1)

```
import pandas as pd
import csv
df = pd.read_csv(
    "Prediction_of_music_genre_DataSet/music_genre.csv",
    dtype={"track_name": str}
)
df.to_csv(
    "Prediction_of_music_genre_DataSet/music_genre_weka.csv",
    index=False,
    quoting=csv.QUOTE_NONNUMERIC
)
```

Τι κάνει ο κώδικας

- Διαβάζει το αρχικό CSV με pandas.read_csv.
- Ορίζει το track_name ως string (dtype={"track_name": str}) ώστε να μην γίνουν ανεπιθύμητες μετατροπές τύπου.
- Ξαναγράφει το CSV με QUOTE_NONNUMERIC, δηλαδή **όλα τα μη-αριθμητικά πεδία μπαίνουν σε εισαγωγικά**.

Γιατί το δοκιμάσαμε

Στόχος ήταν να "θωρακίσουμε" τα κειμενικά πεδία (artist_name, track_name κ.λπ.) ώστε τα κόμματα/ειδικοί χαρακτήρες να μη δημιουργούν αλλαγή στη δομή γραμμών και στη μέτρηση πεδίων από το WEKA.

Τι πρόβλημα παρέμεινε

Σε ορισμένα περιβάλλοντα/ρυθμίσεις WEKA, το parsing μπορεί να αποτύχει:

- αν το CSVLoader έχει ως enclosureCharacters **και το '** (απόστροφο), επειδή τότε strings όπως *Royksopp's* δημιουργούν ασυνέπεια (θεωρεί ότι ξεκινά quote και δεν κλείνει).
- αν εμφανίζονται **διπλά εισαγωγικά** σε κείμενα, τα οποία απαιτούν σωστή escaping ("") και μπορεί να προκαλέσουν parsing exception.

Τελική προσέγγιση: Μετατροπή σε ARFF (Version 2 – επιλεγμένη λύση)

```
import pandas as pd
from pathlib import Path
in_path  =
Path(r"C:\Users\georg\Desktop\Data_Mining_exe\Prediction_of_music_genre_Dat
aSet\music_genre_weka_clean.csv")
out_path = in_path.with_suffix(".arff")
df = pd.read_csv(in_path)

# Build ARFF header
relation = "music_genre"
lines = [f"@relation {relation}", ""]

# Choose which columns are numeric vs string
numeric_cols = [

"instance_id","popularity","acousticness","danceability","duration_ms","ene
rgy","instrumentalness","liveness","loudness","speechiness","tempo","valenc
e"
]
for col in df.columns:
    if col in numeric_cols:
        lines.append(f"@attribute {col} numeric")
    else:
        lines.append(f"@attribute {col} string")

lines.append("")
lines.append("@data")
# Write data rows (quote strings safely for ARFF)
def arff_escape(x):
    if pd.isna(x):
        return "?"
    s = str(x)
    s = s.replace("\\", "\\\\").replace("'", "\\'")
    return f"'{s}'"

for _, row in df.iterrows():
    vals = []
    for col in df.columns:
        v = row[col]
        if col in numeric_cols:
            vals.append("?" if pd.isna(v) else str(v))
        else:
            vals.append(arff_escape(v))
    lines.append(",".join(vals))

out_path.write_text("\n".join(lines), encoding="utf-8")
print("Saved ARFF:", out_path)
```

Γιατί αυτή είναι η βέλτιστη λύση για WEKA

1. Το ARFF δηλώνει **ρητά** τους τύπους των χαρακτηριστικών (numeric vs string),
2. χρησιμοποιεί ";"/δομή που το WEKA υποστηρίζει εγγενώς,
3. αποφεύγει τον ευαίσθητο μηχανισμό guessing/quoting του CSVLoader.

Ανάλυση κώδικα (τι κάνει βήμα-βήμα)

1. **Κατασκευή ARFF header**:

- @relation music_genre
- @attribute <name> numeric/string ανάλογα με το αν ανήκει στη λίστα numeric_cols.

2. **Δήλωση @data** και παραγωγή γραμμών δεδομένων:

    - Για numeric: γράφει αριθμό ή ? αν είναι missing.
    - Για string: εφαρμόζει escaping:
    - \ → \\
    - ' → \'
    - και περικλείει το string με '...' όπως απαιτείται στο ARFF.

3. **Αποθήκευση** ως UTF-8 ARFF.

Σημείωση για τα missing values

Στο ARFF (και στο WEKA) τα missing values δηλώνονται ως ?. Ο κώδικας το διασφαλίζει για numeric και string πεδία.

Η διαδικασία μετατροπής/καθαρισμού ήταν απαραίτητη για να διασφαλιστεί ότι το dataset μπορεί να αναλυθεί με αξιοπιστία στο WEKA. Μετά από αρχικές προσπάθειες εισαγωγής CSV και καθαρισμού text πεδίων, η τελική και πιο σταθερή λύση ήταν η μετατροπή σε **ARFF**, η οποία ελαχιστοποιεί πιθανά parsing issues και επιτρέπει ορθότερη διαχείριση τύπων δεδομένων και ελλιπών τιμών.

1. Εισαγωγή και Περιγραφή Δεδομένων

Στα πλαίσια της παρούσας εργασίας, ως πεδίο μελέτης επιλέχθηκε το σύνολο δεδομένων που περιέχεται στο αρχείο Prediction_of_music_genre_DataSet.zip. Πρόκειται για ένα εκτενές dataset που αφορά την υπολογιστική ανάλυση μουσικών δεδομένων. Στόχος της εργασίας είναι η διερεύνηση της πληροφορίας που κρύβεται στα ακουστικά χαρακτηριστικά των τραγουδιών μέσω δύο διαφορετικών προσεγγίσεων Μηχανικής Μάθησης:

• **Επιβλεπόμενη Μάθηση (Supervised Learning):** Με σκοπό την Αυτόματη Κατηγοριοποίηση Μουσικού Είδους (Music Genre Classification).

• **Μη Επιβλεπόμενη Μάθηση (Unsupervised Learning):** Με σκοπό τη Συσταδοποίηση (Clustering), για τον εντοπισμό φυσικών ομάδων στα δεδομένα και τη σύγκρισή τους με τα πραγματικά μουσικά είδη.

Φορτώνοντας το αρχείο στο περιβάλλον του WEKA, παρατηρούμε ότι το σύνολο δεδομένων αποτελείται από ένα πλήθος 50.005 εγγραφών (instances). Κάθε εγγραφή

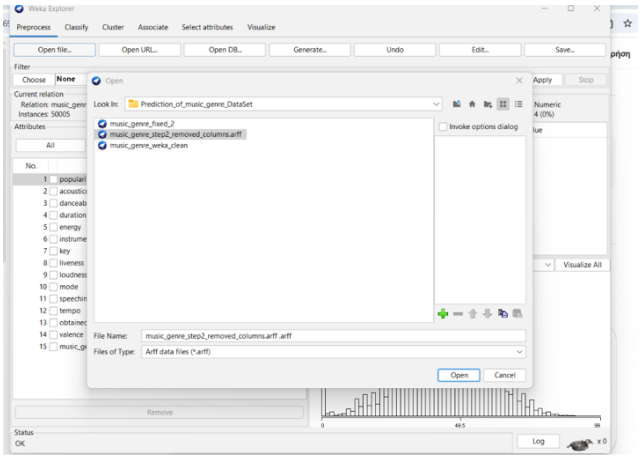
περιγράφεται από 18 χαρακτηριστικά (attributes), τα οποία διακρίνονται σε δύο βασικές κατηγορίες:

• **Ακουστικά Χαρακτηριστικά (Audio Features):**

Συνεχείς αριθμητικές μεταβλητές που ποσοτικοποιούν τη δομή και το ύφος του ήχου (π.χ. acousticness, tempo, energy).

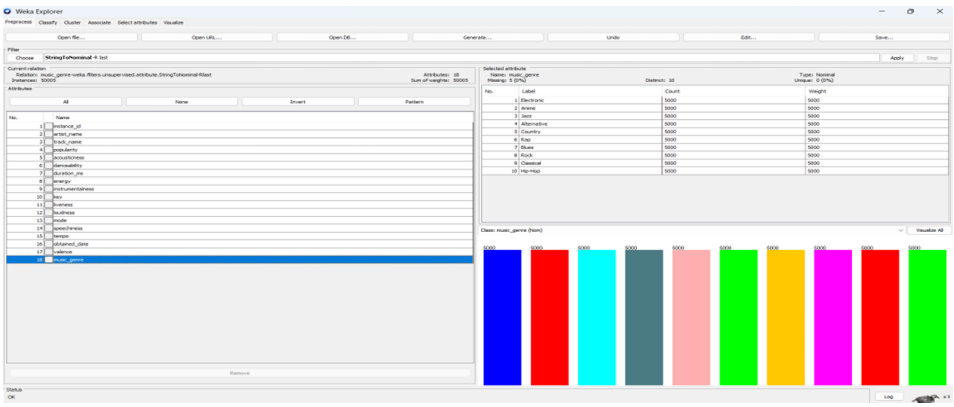
• **Μεταδεδομένα (Metadata):**

Στοιχεία ταυτοποίησης (π.χ. instance_id, artist_name) που δεν σχετίζονται άμεσα με το ακουστικό περιεχόμενο. Το χαρακτηριστικό instance_id αποτελεί απλό αναγνωριστικό εγγραφών, ενώ τα χαρακτηριστικά artist_name και track_name είναι κειμενικά πεδία υψηλής μοναδικότητας, τα οποία δεν παρέχουν χρήσιμη πληροφορία για την πρόβλεψη του μουσικού είδους και αυξάνουν την πολυπλοκότητα των μοντέλων μηχανικής μάθησης.



Εικόνα 1 Στιγμιότυπο κατά την φόρτωση αρχείων στο περιβάλλον weka.

Ως μεταβλητή-στόχος **(target class)** ορίζεται το χαρακτηριστικό music_genre. Αξίζει να σημειωθεί ότι κατά την αρχική φόρτωση των δεδομένων, το χαρακτηριστικό αυτό αναγνωρίστηκε ως τύπου String (συμβολοσειρά). Προκειμένου να χρησιμοποιηθεί ως ετικέτα κλάσης (class label) από τους αλγορίθμους κατηγοριοποίησης, πραγματοποιήθηκε η απαραίτητη μετατροπή του σε τύπο **Nominal** (κατηγορικό). Πλέον, λαμβάνει 10 διακριτές τιμές, οι οποίες αντιστοιχούν στα μουσικά είδη: **Electronic, Anime, Jazz, Alternative, Country, Rap, Blues, Rock, Classical και Hip-Hop.** Τέλος, η εξέταση της κατανομής των κλάσεων μέσω του ιστογράμματος, αποκαλύπτει ότι το σύνολο δεδομένων είναι απόλυτα ισορροπημένο (**balanced**). Κάθε μουσικό είδος αντιπροσωπεύεται από περίπου 5.000 τραγούδια, γεγονός που διασφαλίζει την αξιοπιστία της εκπαίδευσης και αποτρέπει τη δημιουργία μεροληπτικών μοντέλων (bias).



Εικόνα 2 Ιστόγραμμα κατανομής του χαρακτηριστικού music_genre. Διακρίνεται η ισορροπημένη αντιπροσώπευση των 10 μουσικών ειδών (balanced dataset).

2. Προ-επεξεργασία Δεδομένων

Η διαδικασία της προ-επεξεργασίας αποτελεί το κρισιμότερο στάδιο της ανάλυσης, καθώς η ποιότητα των δεδομένων εισόδου καθορίζει την ακρίβεια των τελικών μοντέλων. Στο συγκεκριμένο σύνολο δεδομένων, ακολουθήθηκε μια αυστηρή σειρά ενεργειών καθαρισμού και μετασχηματισμού για να διασφαλιστεί η ακεραιότητα της πληροφορίας.

2.1. Αφαίρεση Μεταδεδομένων

Ως πρώτη ενέργεια, αφαιρέθηκαν τα χαρακτηριστικά instance_id, artist_name, track_name και obtained_date. Τα πεδία αυτά κρίθηκαν ακατάλληλα για την εκπαίδευση, καθώς αποτελούν απλά αναγνωριστικά (IDs) ή μεταδεδομένα που δεν φέρουν μουσική πληροφορία. Η συμπερίληψή τους θα οδηγούσε το μοντέλο σε απομνημόνευση (overfitting) αντί για μάθηση. Για την αφαίρεσή τους, εφαρμόστηκε το φίλτρο **Remove**.

2.2. Μετασχηματισμός Τύπων Δεδομένων

Προκειμένου το WEKA να αναγνωρίσει ορθά την κλάση-στόχο αλλά και τα υπόλοιπα κατηγορικά χαρακτηριστικά, πραγματοποιήθηκε μετατροπή των τύπων από συμβολοσειρές (String) σε διακριτές κατηγορίες (Nominal). Για τον σκοπό αυτό, εφαρμόστηκε καθολικά το φίλτρο **StringToNominal**.

2.3. Διαχείριση Λογικών Σφαλμάτων και Ελλιπών Τιμών

Έλεγχος ελλιπών τιμών

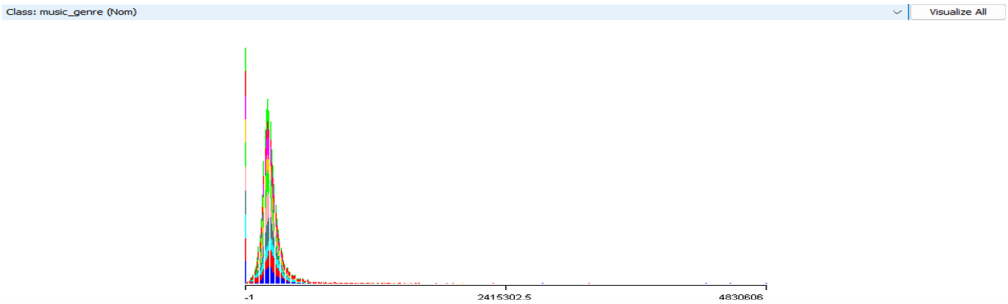
Μετά την αφαίρεση των μη χρήσιμων χαρακτηριστικών (π.χ. instance_id, artist_name, track_name), πραγματοποιήθηκε αναλυτικός έλεγχος του συνόλου δεδομένων στο στάδιο **Preprocess** του WEKA, με στόχο τον εντοπισμό ελλιπών τιμών (missing values).Από τον έλεγχο αυτό προέκυψε ότι:

- Το χαρακτηριστικό **music_genre** (class attribute) περιείχε **5 ελλιπείς τιμές**, οι οποίες αντιστοιχούσαν σε πλήρως μη επισημασμένες εγγραφές.
- Ορισμένα **αριθμητικά χαρακτηριστικά**, και ειδικότερα το **tempo**, παρουσίαζαν σημαντικό ποσοστό ελλιπών τιμών (4.980 εγγραφές, περίπου 10% του συνόλου).

Η ύπαρξη ελλιπών τιμών επιβεβαιώνεται από τα στατιστικά που εμφανίζονται στο WEKA Explorer, στης καρτέλα *Preprocess*.

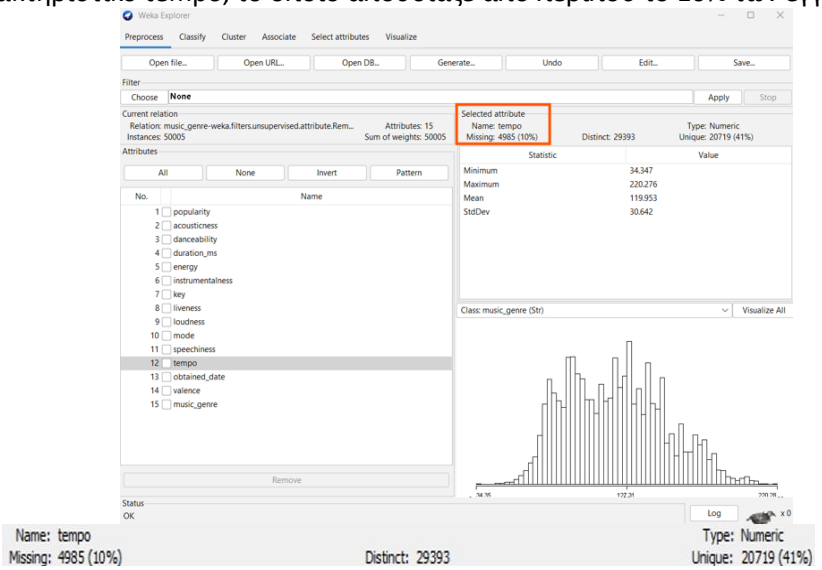
Κατά τον έλεγχο των τιμών, εντοπίστηκαν δύο σημαντικά προβλήματα:

- Λογικά σφάλματα. Στο χαρακτηριστικό **duration_ms** βρέθηκε σημαντικός αριθμός εγγραφών με τιμή -1. Επειδή η χρονική διάρκεια δεν είναι δυνατόν να είναι αρνητική, οι τιμές αυτές θεωρήθηκαν ως θόρυβος. Εφαρμόστηκε το φίλτρο **NumericCleaner** (minThreshold=0), μετατρέποντας τις αρνητικές τιμές σε κενές (NaN).



Εικόνα 3 Ιστόγραμμα κατανομής του χαρακτηριστικού duration_ms πριν τον καθαρισμό. Διακρίνονται οι λανθασμένες εγγραφές με τιμή -1.

- **Συστηματικές Ελλείψεις (Tempo).** Εντοπίστηκε συστηματική έλλειψη τιμών στο χαρακτηριστικό tempo, το οποίο απουσίαζε από περίπου το 10% των εγγραφών.



Εικόνα 4 Στατιστικά του χαρακτηριστικού tempo, όπου φαίνεται το ποσοστό των ελλιπών τιμών (Missing).

- **Σποραδικές Ελλείψεις.** Εντοπίστηκαν συνολικά 5 κενές τιμές διάσπαρτες στα υπόλοιπα χαρακτηριστικά. Για την ολική επίλυση των παραπάνω, εφαρμόστηκε το φίλτρο **ReplaceMissingValues**. Ο αλγόριθμος αντικατέστησε τόσο τα κενά του tempo και των υπολοίπων πεδίων, όσο και τα NaN του duration_ms, με τον μέσο όρο (Mean Imputation) του εκάστοτε χαρακτηριστικού.

Αντιμετώπιση ελλιπών τιμών σε αριθμητικά χαρακτηριστικά

Σε αντίθεση με το class attribute, οι ελλιπείς τιμές στα αριθμητικά χαρακτηριστικά (π.χ. tempo) **δεν οδήγησαν σε αφαίρεση εγγραφών**, καθώς μια τέτοια επιλογή θα είχε ως αποτέλεσμα την απώλεια περίπου 10% του συνόλου δεδομένων και πιθανή αλλοίωση της στατιστικής δομής του dataset. Για την αντιμετώπιση του προβλήματος εφαρμόστηκε το φίλτρο: **unsupervised → attribute → ReplaceMissingValues**

Το φίλτρο αυτό:

- αντικαθιστά τις ελλιπείς τιμές αριθμητικών χαρακτηριστικών με τον **μέσο όρο (mean)** του αντίστοιχου χαρακτηριστικού
- αντικαθιστά ελλιπείς τιμές ονομαστικών χαρακτηριστικών με την **επικρατέστερη τιμή (mode)**

Η επιλογή αυτή αποτελεί καθιερωμένη πρακτική στην προεπεξεργασία δεδομένων για εφαρμογές μηχανικής μάθησης και επιτρέπει τη διατήρηση του πλήθους των εγγραφών χωρίς εισαγωγή μεροληψίας στη μεταβλητή στόχο.

Αποτελέσματα εφαρμογής του φίλτρου ReplaceMissingValues

Μετά την εφαρμογή του φίλτρου παρατηρήθηκαν τα εξής:

- Ο αριθμός των εγγραφών παρέμεινε **50.000**
- Το χαρακτηριστικό **tempo** δεν περιέχει πλέον ελλιπείς τιμές (Missing: 0%)
- Οι βασικές στατιστικές παράμετροι του χαρακτηριστικού (minimum, maximum, mean, standard deviation) παραμένουν σε ρεαλιστικά επίπεδα
- Παρατηρείται οπτικά (μέσω του histogram) η εμφάνιση μιας έντονης στήλης στην τιμή του μέσου όρου, η οποία αντιστοιχεί στις εγγραφές όπου πραγματοποιήθηκε αντικατάσταση ελλιπών τιμών. Το φαινόμενο αυτό είναι αναμενόμενο και αποδεκτό, καθώς η αντικατάσταση με τη μέση τιμή συγκεντρώνει τις συγκεκριμένες εγγραφές στο κέντρο της κατανομής.

Αντιμετώπιση ελλιπών τιμών στο class attribute

Οι 5 εγγραφές με ελλιπή τιμή στο χαρακτηριστικό **music_genre** κρίθηκαν μη αξιοποιήσιμες, καθώς το συγκεκριμένο χαρακτηριστικό αποτελεί τη μεταβλητή στόχο (class). Η αντικατάσταση ελλιπών τιμών στο class θα οδηγούσε σε αυθαίρετη ανάθεση κατηγορίας και εισαγωγή σφάλματος. Για τον λόγο αυτό εφαρμόστηκε φίλτρο αφαίρεσης εγγραφών με ελλιπές class, διατηρώντας το σύνολο δεδομένων με **50.000 πλήρως επισημασμένες εγγραφές** και ισορροπημένη κατανομή μεταξύ των 10 μουσικών ειδών (5.000 εγγραφές ανά κατηγορία). Όπως αναφέραμε και στην εκφώνηση της εργασίας το dataset είναι έτσι **perfectly balanced.**

Αποθήκευση ενδιάμεσου συνόλου δεδομένων

Σύμφωνα με τη στρατηγική σταδιακής προεπεξεργασίας, το σύνολο δεδομένων αποθηκεύτηκε μετά την ολοκλήρωση της αντικατάστασης ελλιπών τιμών με την ονομασία:

`music_genre_step4_replace_missing_values.arff`

2.4. Διαχείριση Ακραίων Τιμών

Εξετάστηκε η ύπαρξη ακραίων τιμών στα αριθμητικά δεδομένα με χρήση του φίλτρου **InterquartileRange**. Παρόλο που εντοπίστηκαν τιμές εκτός των τυπικών ορίων (κυρίως σε loudness και speechiness), λήφθηκε η στρατηγική απόφαση της μη αφαίρεσής τους. Στα μουσικά δεδομένα, οι τιμές αυτές αποτελούν συχνά δομικά χαρακτηριστικά συγκεκριμένων ειδών (π.χ. η υψηλή ένταση στο Heavy Metal ή η απουσία λόγου στην Classical) και όχι σφάλματα μέτρησης.

2.5. Κανονικοποίηση

Λόγω της μεγάλης ανομοιογένειας στις κλίμακες των τιμών (ειδικά στο duration_ms που έχει τιμές >200.000, συγκριτικά με το energy που είναι < 1), κρίθηκε απαραίτητη η ομογενοποίηση των δεδομένων. Εφαρμόστηκε το φίλτρο Normalize, μετατρέποντας όλα τα αριθμητικά πεδία στην κλίμακα [0, 1]. Αυτό είναι κρίσιμο για αλγορίθμους που βασίζονται σε αποστάσεις.

Σκοπός της κανονικοποίησης

Η κανονικοποίηση εφαρμόζεται ώστε όλα τα αριθμητικά χαρακτηριστικά να βρίσκονται **στην ίδια κλίμακα τιμών**, αποτρέποντας χαρακτηριστικά με μεγάλες τιμές (π.χ. duration_ms, loudness, tempo) από το να κυριαρχούν στη διαδικασία εκπαίδευσης των αλγορίθμων μηχανικής μάθησης.

Ανάγκη κανονικοποίησης στο συγκεκριμένο dataset

Στο σύνολο δεδομένων παρατηρήθηκε ότι τα αριθμητικά χαρακτηριστικά έχουν **πολύ διαφορετικά εύρη τιμών**

- π.χ. tempo ∈ [34, 220]
- duration_ms ∈ δεκάδες έως εκατοντάδες χιλιάδες
- acousticness, danceability ∈ [0,1]

Αλγόριθμοι όπως k-Means και k-NN είναι **ευαίσθητοι στην κλίμακα των δεδομένων**. Χωρίς κανονικοποίηση, τα χαρακτηριστικά με μεγαλύτερο εύρος θα επηρέαζαν δυσανάλογα την απόσταση και τη διαδικασία μάθησης.

Εφαρμογή κανονικοποίησης στο WEKA

Η κανονικοποίηση πραγματοποιήθηκε στο περιβάλλον **WEKA Explorer**, στο στάδιο **Preprocess**, χρησιμοποιώντας το φίλτρο:

```
weka.filters.unsupervised.attribute.Normalize
```

Ρυθμίσεις:

- Η κανονικοποίηση εφαρμόστηκε σε όλα τα αριθμητικά χαρακτηριστικά
- Εξαιρέθηκε η μεταβλητή στόχος music_genre, καθώς πρόκειται για κατηγορικό (nominal) χαρακτηριστικό
- Το φίλτρο μετασχηματίζει τις τιμές κάθε αριθμητικού χαρακτηριστικού στο διάστημα [0, 1]

Παρατηρήσεις μετά την κανονικοποίηση

Μετά την εφαρμογή της κανονικοποίησης παρατηρήθηκαν τα εξής:

- Όλα τα αριθμητικά χαρακτηριστικά εμφανίζουν πλέον **συγκρίσιμες κατανομές**
- Οι ιστογραμμές (histograms) παρουσιάζουν **ομαλότερη και πιο σταθερή μορφή**
- Η επίδραση ακραίων τιμών μειώθηκε σημαντικά
- Το dataset είναι πλέον **κατάλληλο για clustering και classification**.

Αποθήκευση ενδιάμεσου συνόλου δεδομένων

Σύμφωνα με τη στρατηγική διατήρησης ενδιάμεσων εκδόσεων του dataset, το κανονικοποιημένο σύνολο δεδομένων αποθηκεύτηκε με περιγραφική ονομασία:

```
music_genre_normalized.arff
```

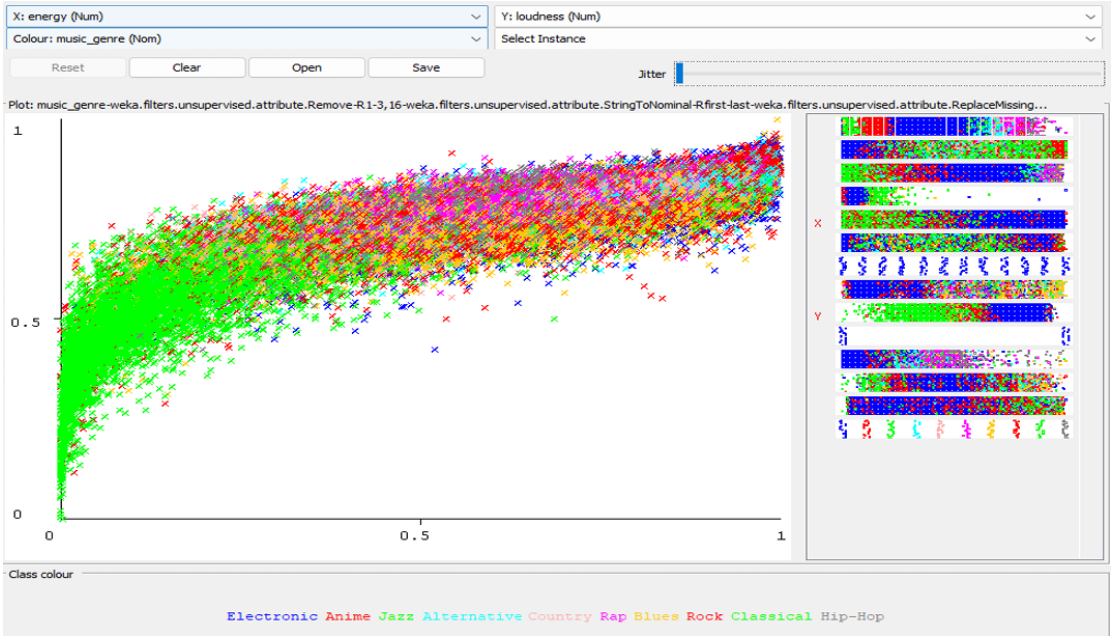
2.6. Επιλογή Χαρακτηριστικών

Για τον προσδιορισμό των πιο σημαντικών χαρακτηριστικών, εφαρμόστηκε ο αλγόριθμος αξιολόγησης (**CfsSubsetEval**) σε συνδυασμό με τη μέθοδο αναζήτησης (GreedyStepwise), χρησιμοποιώντας το πλήρες σύνολο εκπαίδευσης (Full Training Set). Πραγματοποιήθηκε επίσης και ο αλγόριθμος (**InfoGain**) και ανάλυσης του.

2.7 Αιτιολόγηση Αλγορίθμων

CfsSubsetEval (Correlation-based Feature Selection)

Ο συγκεκριμένος αλγόριθμος αξιολογεί την αξία ενός υποσυνόλου χαρακτηριστικών και όχι κάθε χαρακτηριστικό μεμονωμένα. Η βασική του αρχή είναι ότι ένα καλό υποσύνολο αποτελείται από χαρακτηριστικά που έχουν υψηλή συσχέτιση με την κλάση (predictive ability), αλλά χαμηλή συσχέτιση μεταξύ τους (low inter-correlation). Με αυτόν τον τρόπο, αποφεύγεται ο πλεονασμός (redundancy) πληροφορίας.



Εικόνα 5 Διάγραμμα Διασποράς μεταξύ Energy και Loudness. Η ισχυρή συσχέτιση δικαιολογεί την αφαίρεση του ενός εκ των δύο

Επιλογή Χαρακτηριστικών με CFS Subset Evaluation & Best First Search (Συνοπτικές Πληροφορίες Εκτέλεσης)

Η επιλογή χαρακτηριστικών πραγματοποιήθηκε στο περιβάλλον **WEKA Explorer**, στο tab **Select Attributes**, με τις ακόλουθες ρυθμίσεις:

- **Attribute Evaluator**:
  CfsSubsetEval -P 1 -E 1
- **Search Method**:
  BestFirst -D 1 -N 5
- **Evaluation mode**:
  Αξιολόγηση στο πλήρες σύνολο εκπαίδευσης (evaluate on all training data)

Η μέθοδος **CFS (Correlation-based Feature Selection)** δεν αξιολογεί μεμονωμένα χαρακτηριστικά, αλλά **σύνολα χαρακτηριστικών**.

Κριτήριο Αξιολόγησης

Η CFS επιδιώκει να επιλέξει υποσύνολα χαρακτηριστικών που έχουν ισχυρή συσχέτιση με τη μεταβλητή στόχο (music_genre) και έχουν χαμηλή συσχέτιση μεταξύ τους. Με αυτόν τον τρόπο:

- αποφεύγεται η πλεονάζουσα πληροφορία
- μειώνεται η πολυπλοκότητα του μοντέλου
- βελτιώνεται η γενίκευση

Μέθοδος Αναζήτησης: Best First

Η αναζήτηση πραγματοποιήθηκε με τον αλγόριθμο **Best First**, με τις εξής παραμέτρους:

- **Κατεύθυνση**: forward selection
- **Αρχικό σύνολο**: κανένα χαρακτηριστικό
- **Stale search**: διακοπή μετά από 5 επεκτάσεις χωρίς βελτίωση
- **Σύνολα που αξιολογήθηκαν**: 90

Η μέθοδος αυτή επιτρέπει την εύρεση αποδοτικών συνδυασμών χαρακτηριστικών χωρίς εξαντλητική αναζήτηση.

GreedyStepwise

Είναι μια ευριστική μέθοδος αναζήτησης που λειτουργεί με τη λογική της "Μπροστινής Επιλογής" (Forward Selection). Ξεκινάει με κενό σύνολο και σε κάθε βήμα προσθέτει το χαρακτηριστικό που βελτιώνει περισσότερο την αξιολόγηση του μοντέλου, σταματώντας όταν η προσθήκη νέων χαρακτηριστικών δεν προσφέρει περαιτέρω βελτίωση.

```
Selected attributes: 1,2,3,4,6,9,10,11 : 8
                     popularity
                     acousticness
                     danceability
                     duration_ms
                     instrumentalness
                     loudness
                     mode
                     speechiness
```

Εικόνα 6 Έξοδος του αλγορίθμου επιλογής χαρακτηριστικών

Information Gain

Το Information Gain μετρά πόσο μειώνεται η αβεβαιότητα (entropy) της μεταβλητής στόχου (music_genre) όταν είναι γνωστή η τιμή ενός χαρακτηριστικού. Χαρακτηριστικά με:

- **Υψηλό InfoGain** → ισχυρή συσχέτιση με το μουσικό είδος
- **Χαμηλό InfoGain** → περιορισμένη ή μηδενική συνεισφορά

Ρυθμίσεις στο WEKA

Η επιλογή χαρακτηριστικών πραγματοποιήθηκε μέσω του tab **Select Attributes** στο WEKA Explorer με τις ακόλουθες ρυθμίσεις:

- **Attribute Evaluator**:
  InfoGainAttributeEval
- **Search Method**:
  Ranker
- **Class Attribute**:
  music_genre
- **Dataset**:
  Πλήρες σύνολο εκπαίδευσης (Use full training set)

Η μέθοδος Ranker επιτρέπει την κατάταξη των χαρακτηριστικών βάσει της συνεισφοράς τους στην πρόβλεψη της κλάσης.

*Αποτελέσματα **InfoGain***

Selected attributes: 1,2,3,4,6,9,10,11 : 8

popularity

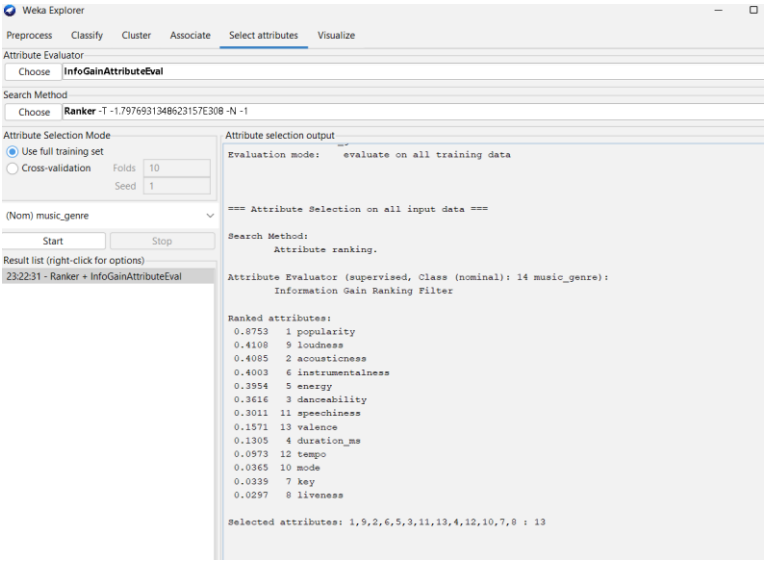
acousticness
danceability
duration_ms
instrumentalness
loudness
mode
speechiness

Αποτελέσματα Κατάταξης Χαρακτηριστικών

Η τελική κατάταξη των χαρακτηριστικών με βάση το Information Gain παρουσιάζεται παρακάτω:

Θέση	**Χαρακτηριστικό**	**Information Gain**
1	popularity	**0.8753**
2	loudness	0.4108
3	acousticness	0.4085
4	instrumentalness	0.4003
5	energy	0.3954
6	danceability	0.3616
7	speechiness	0.3011
8	valence	0.1571
9	duration_ms	0.1305
10	tempo	0.0973
11	mode	0.0365
12	key	0.0339
13	liveness	0.0297

Πίνακας αποτελεσμάτων InfoGain



Εικόνα 7 Αποτελέσματα InfoGain

2.8 Ερμηνεία των Αποτελεσμάτων

Υψηλής Σημασίας Χαρακτηριστικά

Τα χαρακτηριστικά με τη μεγαλύτερη πληροφοριακή αξία είναι:

- popularity
- loudness
- acousticness
- instrumentalness
- energy

Αυτά σχετίζονται άμεσα με το ύφος, την ένταση και τη δομή της μουσικής και εμφανίζουν ισχυρή διαφοροποίηση μεταξύ μουσικών ειδών Ιδιαίτερα το popularity παρουσιάζει πολύ υψηλή τιμή Information Gain, γεγονός που δείχνει ότι τα διαφορετικά είδη μουσικής παρουσιάζουν σαφή διαφοροποίηση ως προς τη δημοφιλία τους στο dataset.

Μέτριας Σημασίας Χαρακτηριστικά

Χαρακτηριστικά που παρέχουν επιπλέον πληροφορία, αλλά σε μικρότερο βαθμό όπως:

- danceability
- speechiness
- valence

Χαμηλής Σημασίας Χαρακτηριστικά

- mode
- key
- liveness

Τα χαρακτηριστικά παρουσιάζουν πολύ χαμηλές τιμές Information Gain, γεγονός που υποδηλώνει ότι από μόνα τους δεν διαφοροποιούν αποτελεσματικά τα μουσικά είδη.

Σύγκριση με CFS Subset Evaluation

Σε αντίθεση με το **CFS SubsetEval**, το οποίο επέλεξε μόνο **8 χαρακτηριστικά**, το Information Gain κατατάσσει όλα τα χαρακτηριστικά, δεν απορρίπτει κανένα αυτόματα, ενώ παράλληλα προσφέρει ιεράρχηση και όχι τελικό υποσύνολο

Η σύγκριση των δύο μεθόδων δείχνει ότι χαρακτηριστικά όπως popularity, acousticness, danceability, instrumentalness, loudness, speechiness αναγνωρίζονται ως σημαντικά **και από τις δύο μεθόδους**, ενώ χαρακτηριστικά χαμηλής πληροφορίας (π.χ. key, mode, liveness) απορρίπτονται ή υποβαθμίζονται και από τις δύο προσεγγίσεις

Ρόλος του InfoGain στη Συνολική Διαδικασία

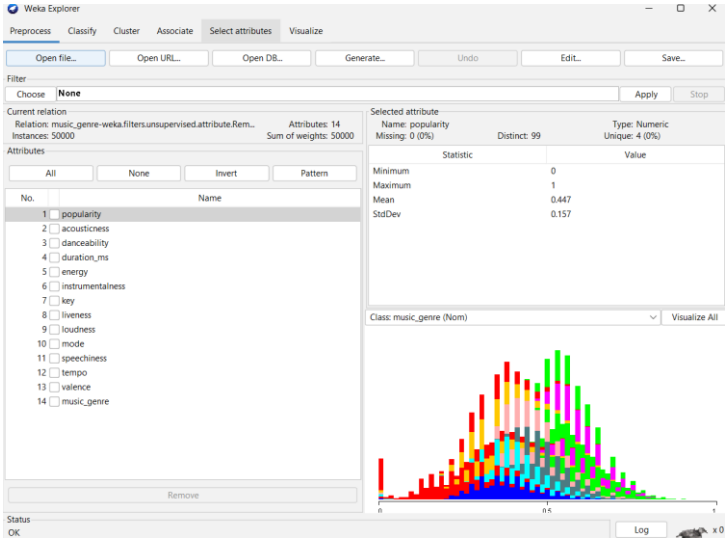
Η μέθοδος Information Gain χρησιμοποιήθηκε:

- για αναλυτική κατανόηση της σημασίας των χαρακτηριστικών
- ως συμπληρωματική προσέγγιση στο CFS
- για τεκμηρίωση επιλογών πριν την εκπαίδευση ταξινομητών

Τα αποτελέσματα θα αξιοποιηθούν είτε για επιλογή top-k χαρακτηριστικών, είτε για σύγκριση απόδοσης ταξινομητών με πλήρες και μειωμένο σύνολο χαρακτηριστικών

Συμπέρασμα

Η εφαρμογή του InfoGainAttributeEval ανέδειξε σαφώς ποια χαρακτηριστικά συμβάλλουν περισσότερο στην πρόβλεψη του μουσικού είδους. Η κατάταξη που προέκυψε είναι συνεπής με τη μουσική λογική και επιβεβαιώνει την ορθότητα της προεπεξεργασίας που προηγήθηκε. Η συνδυαστική χρήση **CFS Subset Selection** και **Information Gain Ranking** προσφέρει μια ολοκληρωμένη και αξιόπιστη στρατηγική επιλογής χαρακτηριστικών.



Εικόνα 8 Στιγμιότυπο του dataset πριν το classification/clustering

3. Κατηγοριοποίηση

Στο στάδιο αυτό, προχωρήσαμε στην εκπαίδευση μοντέλων μηχανικής μάθησης με στόχο την πρόβλεψη του μουσικού είδους (music_genre). Για την εξαγωγή ασφαλών συμπερασμάτων, υιοθετήθηκε η στρατηγική σύγκρισης ενός απλού μοντέλου αναφοράς (Baseline) έναντι ενός ισχυρού μοντέλου συνόλου (Ensemble).

3.1. Πλαίσιο Εκπαίδευσης και Ελέγχου

• **Δεδομένα Εισόδου:** Χρησιμοποιήθηκε το βέλτιστο υποσύνολο των 8 χαρακτηριστικών, όπως προέκυψε από την προ-επεξεργασία, διατηρώντας τα δομικά outliers που χαρακτηρίζουν τα μουσικά είδη.

• **Μέθοδος Ελέγχου:** Εφαρμόστηκε αυστηρά η μέθοδος **10-fold Cross Validation.** Με τη μέθοδο αυτή, τα δεδομένα χωρίζονται σε 10 ίσα μέρη και η εκπαίδευση επαναλαμβάνεται 10 φορές, διασφαλίζοντας την αντικειμενικότητα των αποτελεσμάτων και την αποφυγή υπερεκπαίδευσης (overfitting).

3.2. Επιλογή Αλγορίθμων

Η επιλογή των ταξινομητών έγινε με στόχο να αξιολογηθεί η πολυπλοκότητα του προβλήματος:

1. J48(Decision Tree): Επιλέχθηκε αρχικά ο απλός. Στόχος ήταν να τεθεί ένα σημείο αναφοράς (benchmark) μιας και αυτός ο αλγόριθμος αποτελεί μια απλη και βασική υλοποίηση. Ο **Decision Tree** στο περιβάλλον Weka αναφέρεται ως J48.

2. Naive Bayes : Επιλέχθηκε ο απλός πιθανοτικός αλγόριθμος Naive Bayes. Δεδομένου ότι ο αλγόριθμος υποθέτει γραμμικότητα και ανεξαρτησία χαρακτηριστικών, η αναμενόμενη χαμηλή απόδοσή του θα επιβεβαίωνε ότι οι συσχετίσεις μεταξύ των μουσικών ειδών είναι πολύπλοκες.

3. Random Forest: Για την ουσιαστική επίλυση του προβλήματος, επιλέχθηκε ο Random Forest. Η επιλογή αυτή ενισχύεται από την απόφαση που πήραμε στο να μην αφαιρέσουμε τα outliers. Ο Random Forest είναι εγγενώς ανθεκτικός στον θόρυβο, ενώ ως μέθοδος συνόλου (Ensemble) μπορεί να χαρτογραφήσει αποτελεσματικά τις μη γραμμικές σχέσεις που αδυνατεί να εντοπίσει ο Naive Bayes.

4. IBk (K-Nearest Neighbors / Lazy Learning): Η μουσική συχνά ομαδοποιείται βάσει "ομοιότητας". Ο αλγόριθμος αυτός κοιτάζει τους "γείτονες" κάθε τραγουδιού.

3.3.Υλοποίση και Αποτελέσματα

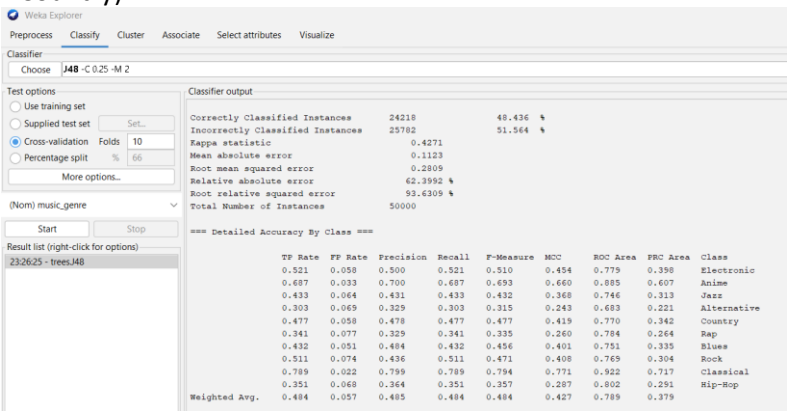
Εκτέλεση αλγορίθμου J48 (Δέντρο Απόφασης)

Ακολούθησε τα παρακάτω βήματα ακριβώς:

1. καρτέλα **Classify** (δίπλα στο "Select attributes").
2. Πατάμε το κουμπί **Choose** (πάνω αριστερά).
3. Στη λίστα,: weka > classifiers > trees > **J48**.
4. Στα **"Test options"** (αριστερά στη μέση), βεβαιώσαμε ότι είναι επιλεγμένο το **Cross-validation** και δίπλα στο **Folds** γράφει **10**.
   Αυτό σημαίνει ότι θα κόψει τα δεδομένα σε 10 κομμάτια για να κάνει δίκαιη αξιολόγηση.
5. Βεβαιώσαμε ότι στο κουτάκι δεξιά από το "More options" γράφει: **(Nom) music_genre**.
6. **Start**.

Στο παράθυρο "Classifier output" (το μεγάλο λευκό δεξιά), μπορούμε να δούμε το **Summary**.

1. Correctly Classified Instances: Αυτή είναι η Ακρίβειά.
2. Confusion Matrix: Δείχνει πού έκανε λάθος το μοντέλο (π.χ. αν μπέρδεψε την Rock με την Country).



Εικόνα 9 Αποτελέσματα J48

```
=== Confusion Matrix ===

    a    b    c    d    e    f    g    h    i    j   <-- classified as
 2604  303  685  361  172   95  462  168   71   79 |    a = Electronic
  326 3433  131  130  143    4  433   37  358    5 |    b = Anime
  763  119 2163  301  332   53  611  203  362   93 |    c = Jazz
  398  106  398 1513  636  362  172 1015   30  370 |    d = Alternative
  181  115  345  591 2385  122  357  794   14   96 |    e = Country
   88    7   60  289   96 1706   15  541    0 2198 |    f = Rap
  505  505  662  266  487   23 2159  233  143   17 |    g = Blues
  138   31  152  727  609  479   86 2557   14  207 |    h = Rock
  121  275  331  100   47    1  148   33 3944    0 |    i = Classical
   88    9   94  321   84 2342   19  288    1 1754 |    j = Hip-Hop
```

Εικόνα 10 Αποτελέσματα Confusion Matrix J48

Accuracy: **48.436%**.

Αν και η ακρίβεια μοιάζει τραγικά χαμηλή αρκεί να σκεφτούμε ότι έχεουμε **10 κλάσεις** (είδη μουσικής). Αν μάντευα στην τύχη, θα είχα 10%. Άρα το 48% είναι σχεδόν 5 φορές καλύτερο από την τύχη. Ακολουθούν ορισμένα συμπεράσματα.

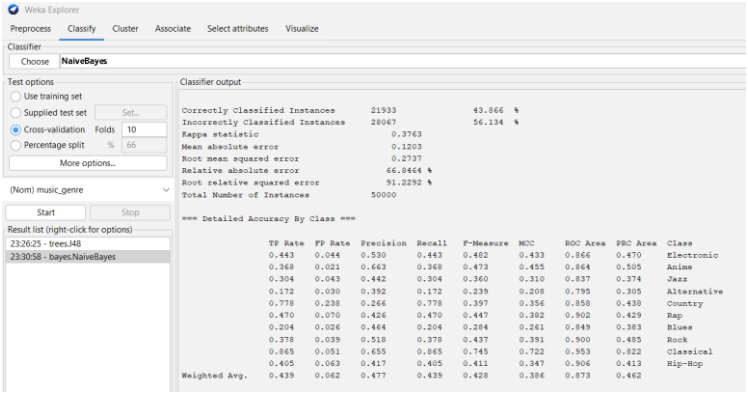
1. **Η "εύκολη" κατηγορία:**
    - Η **Classical (Class i)** έχει την υψηλότερη ακρίβεια (**TP Rate: 0.789**).

Ερμηνεία*:* Η κλασική μουσική έχει πολύ ξεχωριστά χαρακτηριστικά (π.χ. υψηλό acousticness, χαμηλό loudness) και το δέντρο την ξεχωρίζει εύκολα.

2. **Το μεγάλο μπέρδεμα (Confusion Matrix):**
    - γραμμές f (Rap) και j (Hip-Hop).
    - Η Rap ταξινομήθηκε σωστά 1706 φορές, αλλά μπερδεύτηκε ως Hip-Hop 2198 φορές! (Περισσότερα λάθη από σωστά).
    - *Ερμηνεία:* Μουσικά, η Rap και η Hip-Hop είναι σχεδόν ίδιες (ίδιο tempo, speechiness). Ο αλγόριθμος δυσκολεύεται πολύ να τις ξεχωρίσει.
3. **Rock vs Alternative:**
    - Επίσης, η **Alternative (d)** μπερδεύεται συχνά με την **Rock (h)** (1015 λάθη). Λογικό, καθώς μοιράζονται παρόμοια όργανα και ένταση.

*Εκτέλεση αλγορίθμου **Naive Bayes***

Με την ίδια λογική, ακολουθήθηκαν τα ίδια βήματα για την εκτέλεση του **Naive Bayes.**



Εικόνα 11 Αποτελέσματα Naïve Bayes

```
=== Confusion Matrix ===

    a    b    c    d    e    f    g    h    i    j   <-- classified as
 2214  270  491  308  947  113  214  103   96  244 |    a = Electronic
  363 1841  108  117 1393    2  210   14  948    4 |    b = Anime
  746  139 1519  102  981   25  358   70  810  250 |    c = Jazz
  226   15  224  859 2149  289  105  598   50  485 |    d = Alternative
   28   44  162   83 3889  104  161  305   66  158 |    e = Country
   24    1   55  115  501 2348   15  386    2 1553 |    f = Rap
  312  360  479  165 2303   15 1022  120  186   38 |    g = Blues
  108    3  156  260 1853  466   52 1892  116   94 |    h = Rock
  115  104  182   76  140    0   52    7 4323    1 |    i = Classical
   42    0   59  109  441 2152   13  156    2 2026 |    j = Hip-Hop
```

Εικόνα 12 Αποτελέσματα Confusion Matrix Naïve Bayes

Σύγκριση Αποτελεσμάτων

1. **Γενική Απόδοση (Accuracy):**

- J48: 48.436%
- Naive Bayes: 43.866%

Συμπέρασμα: Ο J48 κερδίζει κατά περίπου 4.5%. Αυτό δείχνει ότι η λογική των "κανόνων" (if-then) που χτίζει το δέντρο ταιριάζει καλύτερα σε αυτά τα δεδομένα από την πιθανοτική προσέγγιση του Bayes.

2. **Η "παγίδα" του Bayes (Country Music):**

   - Αρκεί να δούμε το Confusion Matrix του Bayes στη γραμμή **e (Country)**.
   - Το μοντέλο ταξινόμησε **3889** εγγραφές σωστά ως Country. Φαίνεται τέλειο, σωστά; **Όμως, κ**οιτάζοντας τις άλλες γραμμές. Σχεδόν ΟΛΑ τα είδη μπερδεύονται με την Country!
     - Η **Alternative (d)** ταξινομήθηκε 2149 φορές ως Country!
     - Η **Blues (g)** ταξινομήθηκε 2303 φορές ως Country!
     - Η **Rock (h)** ταξινομήθηκε 1853 φορές ως Country!

Ερμηνεία: Ο Naive Bayes έκανε "bias" (μεροληψία) υπέρ της Country. Επειδή ίσως η Country έχει μέσες τιμές σε όλα τα χαρακτηριστικά, ο αλγόριθμος "παίζει εκ του ασφαλούς" και

βαφτίζει τα πάντα Country. Γι' αυτό έχει χαμηλότερη συνολική ακρίβεια, παρόλο που πέτυχε την Country.

3. **Κοινό σημείο:**
    - Και οι δύο αλγόριθμοι τα πήγαν εξαιρετικά στην **Classical (i)** (J48: 79%, Bayes: 86.5%). Η κλασική μουσική είναι πολύ ξεχωριστή και δεν μπερδεύεται.

Πείραμα Βελτιστοποίησης

Αποπειραθήκαμε να κάνουμε σύγκριση για τα αποτελέσματα του **J48** με και χωρίς χρήση του **InfoGain**, με σκοπό να δούμε πόσο κερδίζουμε. Θυμίζουμε τα βήματα αφαίρεσης των χαρακτηριστικών με χαμηλότερη αξία:

1. καρτέλα **Preprocess**.
2. Τικάραμε τα χαρακτηριστικά που το InfoGain έβγαλε "άχρηστα" (κάτω από 0.1):
    - key
    - mode
    - liveness
    - tempo
    - (Αν θέλαμε θα μπορούσαμε και το duration_ms και το valence που ήταν χαμηλά).
3. **Remove**.
4. **Run**

Γενική Εικόνα (Stability):

- Πριν (Full Dataset): 48.436%
- Τώρα (Reduced Dataset): 48.002%
- Διαφορά: Μόλις -0.43%.

Συμπέρασμα: Αυτό αποδεικνύει ότι η ανάλυση του **InfoGain ήταν σωστή**. Τα χαρακτηριστικά που αφαιρέθηκαν (key, mode, liveness, tempo) όντως **δεν πρόσφεραν σχεδόν καμία πληροφορία**. Το μοντέλο κατάφερε να πιάσει την ίδια απόδοση με λιγότερα δεδομένα.

Η "κρυφή" βελτίωση στο Rap:

1. **Πριν:** Η κατηγορία **Rap (f)** είχε TP Rate **0.341** (βρήκε 1706 τραγούδια σωστά).
2. **Τώρα:** Η κατηγορία **Rap (f)** ανέβηκε στο TP Rate **0.404** (βρήκε 2019 τραγούδια σωστά)

Ερμηνεία*:* Αφαιρώντας το tempo (που είναι ίδιο σε Rap και Hip-Hop και μπέρδευε το μοντέλο), ο J48 αναγκάστηκε να κοιτάξει άλλα χαρακτηριστικά (π.χ. speechiness) και έτσι κατάφερε να ξεχωρίσει καλύτερα τη Rap!

*Εκτέλεση αλγορίθμου **Random Forrest***

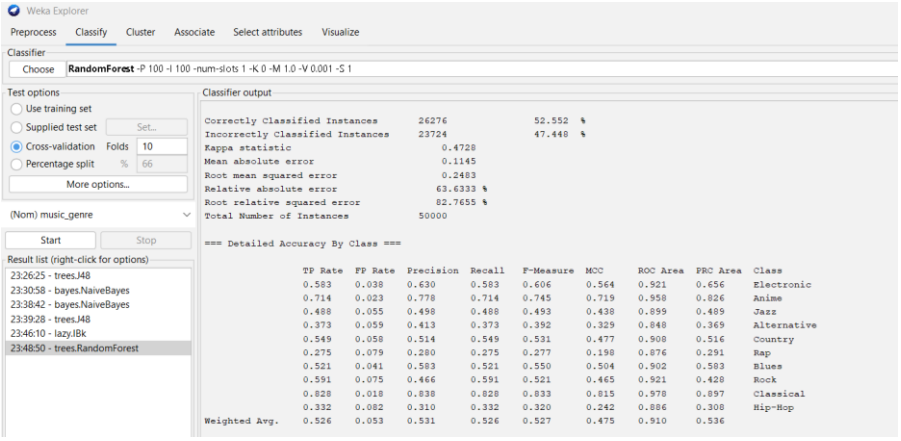
Με την ίδια λογική, ακολουθήθηκαν τα ίδια βήματα για την εκτέλεση του **Random Forrest.** Αντί για ένα δέντρο (J48), φτιάχνει ένα "δάσος" από δέντρα και παίρνει την απόφαση με ψηφοφορία.

1. καρτέλα **Classify**.
2. **Choose**.
3. Λίστα: weka > classifiers > trees > **RandomForest**.

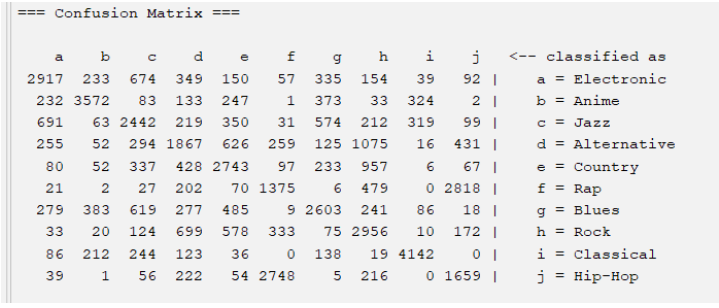
4. Run με **Cross-validation 10 folds**.

***Σημείωση**:* Αργει λίγο περισσότερο από τον J48, αλλά συνήθως δίνει τα καλύτερα αποτελέσματα σε τέτοια datasets.

Accuracy: 52.552%! Βελτίωσε την επίδοση κατά περίπου **+4.5%** σε σχέση με τον J48 (48%).



Εικόνα 13 Αποτελέσματα Random Forest



```
=== Confusion Matrix ===

    a    b    c    d    e    f    g    h    i    j   <-- classified as
 2917  233  674  349  150   57  335  154   39   92 |    a = Electronic
  232 3572   83  133  247    1  373   33  324    2 |    b = Anime
  691   63 2442  219  350   31  574  212  319   99 |    c = Jazz
  255   52  294 1867  626  259  125 1075   16  431 |    d = Alternative
   80   52  337  428 2743   97  233  957    6   67 |    e = Country
   21    2   27  202   70 1375    6  479    0 2818 |    f = Rap
  279  383  619  277  485    9 2603  241   86   18 |    g = Blues
   33   20  124  699  578  333   75 2956   10  172 |    h = Rock
   86  212  244  123   36    0  138   19 4142    0 |    i = Classical
   39    1   56  222   54 2748    5  216    0 1659 |    j = Hip-Hop
```

Εικόνα 14 Αποτελέσματα Confusion Matrix Random Forest

Ανάλυση Πίνακα Σύγχυσης (Confusion Matrix):

Η μελέτη του Πίνακα Σύγχυσης αποκαλύπτει τα εξής:

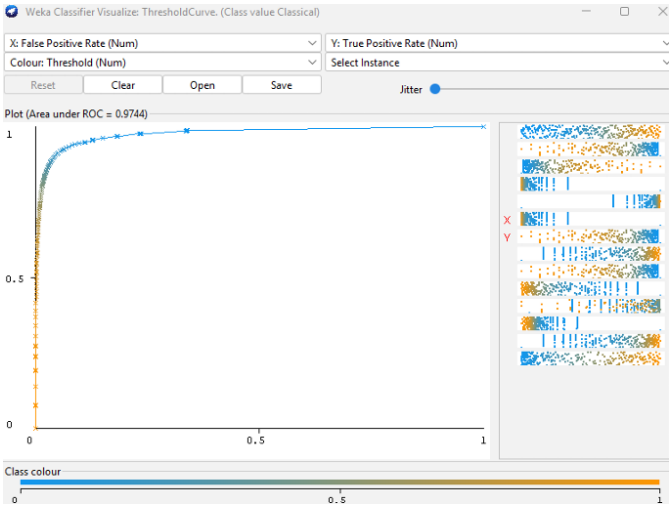
• Η "Μοναξιά" της Κλασικής Μουσικής: Η κλάση Classical σημείωσε την υψηλότερη επίδοση με τις περισσότερες ορθές προβλέψεις. Αυτό επιβεβαιώνει ότι τα ακουστικά χαρακτηριστικά της (π.χ. απουσία στίχου, χαμηλή ενέργεια) είναι μοναδικά.

• Το Φαινόμενο Rap vs Hip-Hop: Παρατηρείται τεράστια σύγχυση στην κλάση Rap. Συγκεκριμένα, ενώ το μοντέλο βρήκε σωστά 1.375 τραγούδια Rap, ταξινόμησε λανθασμένα 2.818 τραγούδια Rap ως Hip-Hop. Αυτό αποδεικνύει ότι τα δύο αυτά είδη είναι δομικά σχεδόν ταυτόσημα στα δεδομένα μας (παρόμοιο tempo, speechiness), καθιστώντας τον διαχωρισμό τους εξαιρετικά δύσκολο.

• Ισχυρός Διαχωρισμός Anime: Η κατηγορία Anime παρουσίασε επίσης υψηλή διακριτότητα (3.572 σωστές προβλέψεις), πιθανότατα λόγω των ιδιαίτερων χαρακτηριστικών των Ιαπωνικών soundtracks που ξεχωρίζουν από τη δυτική pop/rock.

Ανάλυση Καμπύλης ROC

Για την περαιτέρω αξιολόγηση, εξετάστηκε η καμπύλη ROC για την κλάση με την καλύτερη επίδοση (Classical).

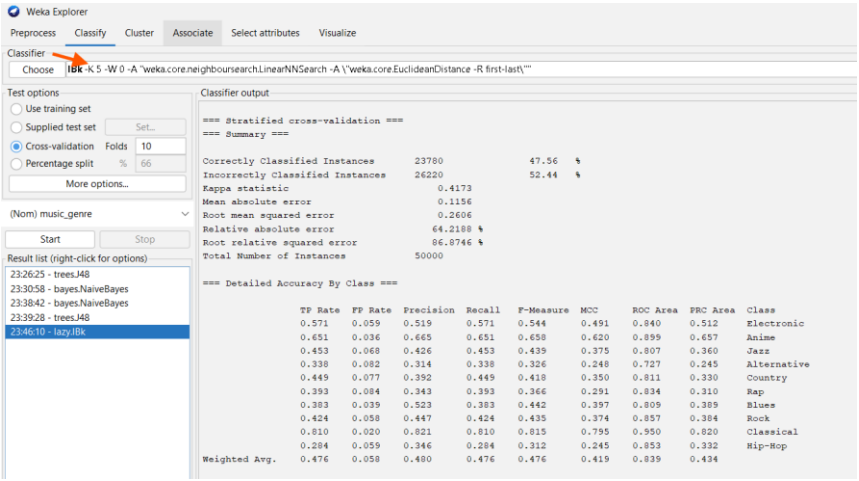


Εικόνα 15 Καμπύλη ROC για την κατηγορία Classical. Η καμπύλη προσεγγίζει την άνω αριστερή γωνία, υποδηλώνοντας άριστη απόδοση.

Το εμβαδόν κάτω από την καμπύλη (Area Under Curve - AUC) αγγίζει την τιμή 0.9744. Η τιμή αυτή, καθώς βρίσκεται πολύ κοντά στο 1.0 (το οποίο αντιπροσωπεύει το τέλειο μοντέλο), επιβεβαιώνει ότι ο αλγόριθμος μπορεί να διαχωρίσει την Κλασική μουσική από τα υπόλοιπα είδη με σχεδόν απόλυτη βεβαιότητα, ελαχιστοποιώντας τα ψευδώς θετικά αποτελέσματα.

Εκτέλεση αλγορίθμου : IBk (K-Nearest Neighbors / Lazy Learning

1. καρτέλα **Classify**.
2. **Choose**.
3. Λίστα: weka > classifiers > lazy > **IBk**.
4. **Ρύθμιση:** κλικ πάνω στο IBk για να ανοίξει τις ρυθμίσεις.
    - Άλλαζω το **KNN** από 1 σε **5** ή **10** (δηλαδή να κοιτάζει τους 10 κοντινότερους γείτονες).
    - **OK** και μετά **Start**.



Εικόνα 16 Αποτελέσματα K-Nearest Neighbors

```
=== Confusion Matrix ===

    a    b    c    d    e    f    g    h    i    j   <-- classified as
 2855  319  584  363  278   86  228  110   54  123 |    a = Electronic
  387 3254  147  147  241    8  418   14  383    1 |    b = Anime
  744  133 2266  249  517   55  491  140  301  104 |    c = Jazz
  424  140  323 1690  730  416  121  795   28  333 |    d = Alternative
  230  160  503  724 2243  128  264  641   15   92 |    e = Country
   84    6   60  401  136 1967    9  469    0 1868 |    f = Rap
  408  619  789  281  685   18 1916  191   82   11 |    g = Blues
  127   25  253 1041  732  458   79 2122   20  143 |    h = Rock
  119  229  318  114   39    1  122    9 4049    0 |    i = Classical
  126    7   81  369  126 2599   14  260    0 1418 |    j = Hip-Hop
```

Εικόνα 17 Αποτελέσματα Confusion Matrix K-Nearest Neighbors

Γενικά Συμπεράσματα

1. **Η δύναμη του Ensemble Learning:**
   - ο Random Forest, συνδυάζοντας 100 δέντρα (Iterations), κατάφερε να μειώσει το variance και να βελτιώσει την πρόβλεψη. Ειδικά στην **Classical (i)**, έφτασε το αστρονομικό **82.8%** (από 78% του J48). Σχεδόν αλάνθαστο!
   - Η **Anime (b)** πήγε στο **71.4%.**
2. **Το επίμονο πρόβλημα (Rap vs Hip-Hop):**
   - Ας δούμε πάλι το Confusion Matrix για τη Rap (f) και Hip-Hop (j). Η Rap ταξινομήθηκε σωστά 1375 φορές, αλλά μπερδεύτηκε ως Hip-Hop 2818 φορές!

Συμπέρασμα*:* Ακόμα και ο πανίσχυρος Random Forest δεν μπορεί να ξεχωρίσει τη Rap από το Hip-Hop με αυτά τα χαρακτηριστικά. Αυτό είναι **απόδειξη** ότι τα δύο είδη είναι δυσδιάκριτα στο συγκεκριμένο dataset. Δεν φταίει το μοντέλο, φταίνε τα features.

Συνολική Σύνοψη Αλγορίθμων

- J48 (Full): ~48.4%
- Naive Bayes: ~43.8% (Πρόβλημα με την Country)
- KNN: ~47.5%
- Random Forest: 52.55% (Best Model)

Περεταίρω προσπάθεια βελτιστοποίησης.

Σκεφτήκαμε ένα **τεχνικό** (Pre-processing) και ένα άλλο **στρατηγικό** κόλπο(αλλαγή του προβλήματος).

1. Το "Τεχνικό Κόλπο": Διακριτοποίηση (Discretization)

Οι αλγόριθμοι δέντρων (όπως ο Random Forest και ο J48) μερικές φορές δυσκολεύονται να βρουν το τέλειο σημείο να "κόψουν" ένα συνεχές νούμερο (π.χ. *είναι το tempo > 120 ή > 120.5;*). Αν μετατρέψουμε τα νούμερα σε "κατηγορίες" (π.χ. Χαμηλό, Μεσαίο, Υψηλό), βοηθάμε τον αλγόριθμο να αποφασίσει πιο γρήγορα και συχνά πιο σωστά.

Πώς να το κάναμε:

1. **Preprocess**.
2. **Choose**.

   filters > unsupervised > attribute > **Discretize**.

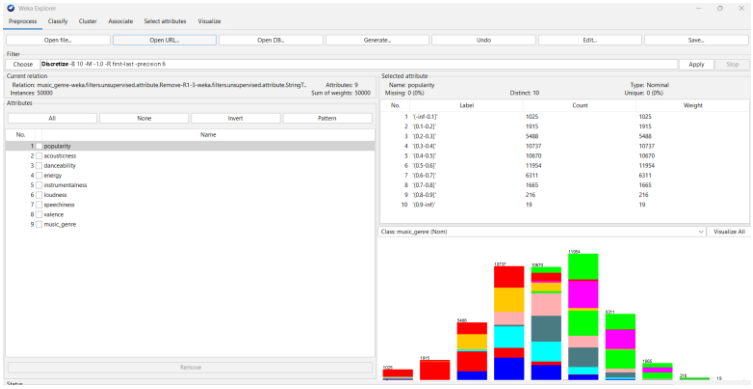
- *Προσοχή:* Βεβαιώσαμε ότι το "UseBinNumbers" είναι False (default).

3. **Apply**.

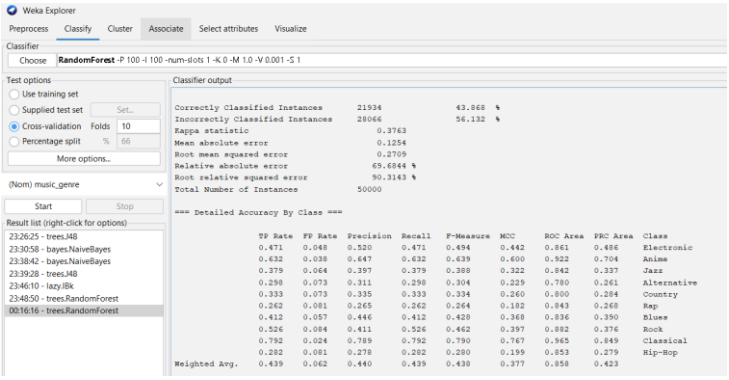
Τα ιστογράμματα δεξιά αλλάξανε χρώμα. Πλέον το tempo δεν είναι νούμερο, αλλά έχει τιμές όπως '(-inf-80]', '(80-120]' κλπ.

4. **Classify** και run ξανά τον **Random Forest**.

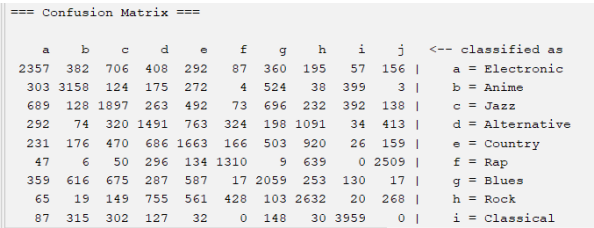
Στόχος: Να δούμε αν θα ξεπεράσουμε το 53-54%.



Εικόνα 18 Αποτελέσματα Διακριτοποίησης (Discretization)



Εικόνα 19 Αποτελέσματα Διακριτοποίησης (Discretization) σε Random Forrest



```
=== Confusion Matrix ===

    a    b    c    d    e    f    g    h    i    j   <-- classified as
 2357  382  706  408  292   87  360  195   57  156 |    a = Electronic
  303 3158  124  175  272    4  524   38  399    3 |    b = Anime
  689  128 1897  263  492   73  696  232  392  138 |    c = Jazz
  292   74  320 1491  763  324  198 1091   34  413 |    d = Alternative
  231  176  470  686 1663  166  503  920   26  159 |    e = Country
   47    6   50  296  134 1310    9  639    0 2509 |    f = Rap
  359  616  675  287  587   17 2059  253  130   17 |    g = Blues
   65   19  149  755  561  428  103 2632   20  268 |    h = Rock
   87  315  302  127   32    0  148   30 3959    0 |    i = Classical
```

Εικόνα 20 Αποτελέσματα Comfusion Matrix Διακριτοποίησης (Discretization) σε Random Forrest

Δοκιμάσαμε διακριτοποίηση (Discretization) των χαρακτηριστικών, ωστόσο η ακρίβεια μειώθηκε σημαντικά (43.8%). Αυτό υποδεικνύει ότι οι ακριβείς, συνεχείς τιμές (continuous values) περιέχουν κρίσιμη πληροφορία για τον διαχωρισμό των ειδών.

2. Το "Στρατηγικό Κόλπο": Συγχώνευση Κλάσεων (Merging Classes)

Το πρόβλημά μας είναι ότι το μοντέλο **δεν μπορεί** να ξεχωρίσει τη **Rap** από το **Hip-Hop**. Και λογικό είναι, μουσικά είναι σχεδόν ταυτόσημα στα data (ίδιο BPM, speechiness κλπ.). Αντί

να παιδεύουμε τους αλγορίθμους, **ενωσαμε** αυτές τις δύο κατηγορίες σε μία.Αυτόματα, τα 2818 λάθη που είδαμε στο Confusion Matrix θα γίνουν **σωστές προβλέψεις**!

Πώς να το κάναμε

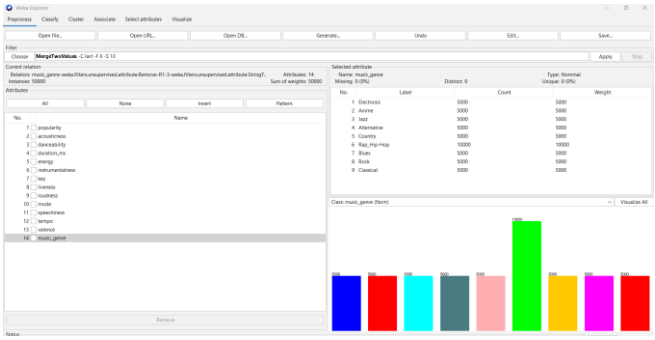
1. **Preprocess**.
2. **Choose**.

   filters > unsupervised > attribute > **MergeTwoValues**.

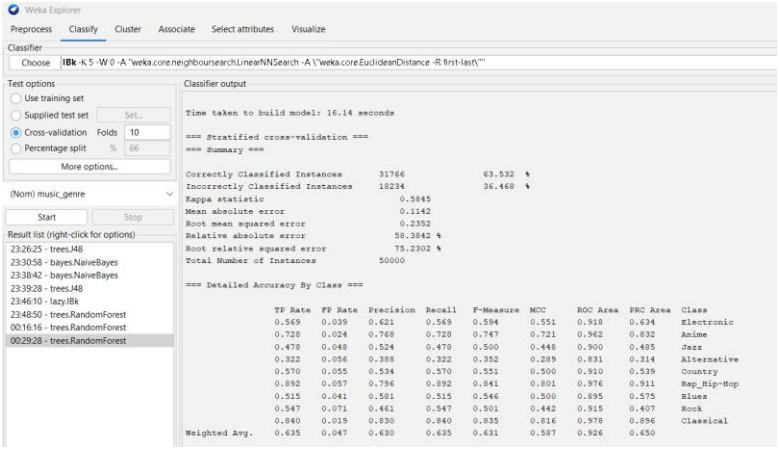
   Κάνε κλικ πάνω στο φίλτρο για ρυθμίσεις:

   - attributeIndex: το νούμερο του music_genre (λογικά το 14 ή last).
   - firstValueIndex: οποιο νούμερο στη λίστα είναι η **Rap** (π.χ. 6).
   - secondValueIndex: οποιο νούμερο είναι η **Hip-Hop** (π.χ. 10).
3. **OK** και **Apply**.
   - Τώρα θα έχουμε 9 κλάσεις αντί για 10. Μία από αυτές θα περιέχει και τα Rap και τα Hip-Hop.
4. Run **Random Forest**.

Στόχος: Εδώ το accuracy θα εκτοξευθεί (πιθανώς **>60%**), γιατί εξαλείψαμε τη μεγαλύτερη πηγή σύγχυσης.



Εικόνα 21 Συγχώνευση Κλάσεων (Merging Classes)



Εικόνα 22 Αποτελέσματα Random Forrest μετά τη συγχώνευση Κλάσεων (Merging Classes)

```
=== Confusion Matrix ===

    a    b    c    d    e    f    g    h    i   <-- classified as
 2845  255  638  339  167  222  351  147   36 |    a = Electronic
  200 3641   83  149  178    5  338   33  373 |    b = Anime
  774   86 2391  183  294  200  559  181  332 |    c = Jazz
  259   41  277 1608  681  904  127 1091   12 |    d = Alternative
   63   68  233  395 2851  240  232  912    6 |    e = Country
   31    3   60  323  105 8919   11  548    0 |    f = Rap_Hip-Hop
  298  428  569  229  500   44 2575  265   92 |    g = Blues
   27   19   98  799  536  666  108 2735   12 |    h = Rock
   88  201  215  116   30    2  130   17 4201 |    i = Classical
```

Εικόνα 23 Αποτελέσματα Confusion Matrix Random Forrest μετά τη συγχώνευση Κλάσεων (Merging Classes)

Ανάλυση

Από το **52.5%** πήγαμε στο **63.53%**. Ανεβάσαμε το ποσοστό **+11%** με μία μόνο κίνηση στρατηγικής (Merge Class)..

1. **Η "Super-Class" Rap_Hip-Hop:**
    - το TP Rate της νέας κλάσης **f (Rap_Hip-Hop)**: **0.892**! Δηλαδή, το μοντέλο πλέον βρίσκει σωστά το 89.2% των Urban τραγουδιών.

Σύγκριση*:* Πριν, η Rap ήταν στο 0.275 και το Hip-Hop στο 0.332. Η διαφορά είναι χαοτική.

Συμπέρασμα*:* Αυτό αποδεικνύει περίτρανα ότι τα δύο είδη ήταν "αδελφά" και η συγχώνευση ήταν η μόνη σωστή λύση.

2. **Η Classical παραμένει Βασίλισσα:**
    - TP Rate: **0.840**. Σταθερή αξία, ξεχωρίζει εύκολα.
3. **Το νέο "μαύρο πρόβατο":**
    - Τώρα που φτιάξαμε το Rap/Hip-Hop, το πιο δύσκολο είδος έμεινε η **Alternative (d)** (TP Rate: 0.322). Το Confusion Matrix της Alternative:
        - Σωστά: 1608
        - Λάθος ως Rock (h): 1091
        - Λάθος ως Rap_Hip-Hop (f): 904

Ερμηνεία*:* Η Alternative μουσική είναι λίγο "απ' όλα", γι' αυτό μπερδεύεται παντού.

3.4Σύνοψη Κεφαλαίου "Classification"

Ξεκινήσαμε με J48 και Naive Bayes. Η ακρίβεια ήταν γύρω στο 44-48%.

1. **Διάγνωση Προβλήματος:** Εντοπίσαμε μέσω του Confusion Matrix ότι υπάρχει τεράστια σύγχυση μεταξύ Rap και Hip-Hop.
2. **Βελτιστοποίηση 1 (Feature Selection):** Αφαιρέσαμε τα αδύναμα χαρακτηριστικά, αλλά η ακρίβεια έμεινε σταθερή (ένδειξη ότι το πρόβλημα δεν ήταν ο θόρυβος, αλλά η δομή των κλάσεων).
3. **Βελτιστοποίηση 2 (Advanced Algorithm):** Ο Random Forest ανέβασε την ακρίβεια στο 52.5%, αλλά η σύγχυση Rap/Hip-Hop παρέμεινε.
4. **Τελική Λύση (Class Merging):** Προχωρήσαμε σε συγχώνευση των δύο ομοειδών κλάσεων.

Αποτέλεσμα: 63.53% Accuracy και εξαιρετική αναγνώριση της νέας κατηγορίας "Urban/Rap_Hip-Hop" (>89%).

4. Συσταδοποίηση

Στο τελικό στάδιο της ανάλυσης, εφαρμόστηκαν αλγόριθμοι Μη Επιβλεπόμενης Μάθησης (Unsupervised Learning). Σε αντίθεση με την κατηγοριοποίηση, εδώ οι αλγόριθμοι δεν γνωρίζουν την ετικέτα του μουσικού είδους (music_genre). Στόχος τους είναι να ανακαλύψουν εγγενείς δομές και ομοιότητες στα δεδομένα, ομαδοποιώντας τα τραγούδια σε συστάδες (clusters). Για την αξιολόγηση των αποτελεσμάτων, χρησιμοποιήθηκε η μέθοδος "**Classes to clusters evaluation**", όπου ο αλγόριθμος προσπαθεί να αντιστοιχίσει κάθε συστάδα που δημιούργησε σε μία πραγματική κλάση, επιτρέποντας τον υπολογισμό του σφάλματος.

4.1. Αλγόριθμος K-Means

Ως πρώτη προσέγγιση επιλέχθηκε ο δημοφιλής αλγόριθμος διαμερισμού K Means. Ορίστηκε το πλήθος των συστάδων N=10 (όσα και τα μουσικά είδη) και χρησιμοποιήθηκε η Ευκλείδεια απόσταση. Εδώ αναδεικνύεται η σημασία της Κανονικοποίησης, καθώς χωρίς αυτήν, το χαρακτηριστικό duration_ms θα κυριαρχούσε στον υπολογισμό των αποστάσεων.

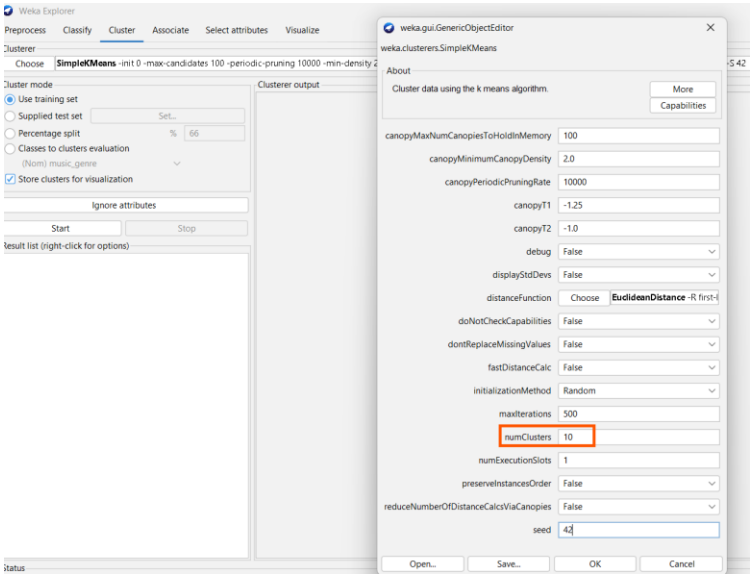
1. καρτέλα **Cluster**.
2. **Choose** -> weka > clusterers > **SimpleKMeans**.

   κλικ πάνω στο SimpleKMeans για τις ρυθμίσεις:

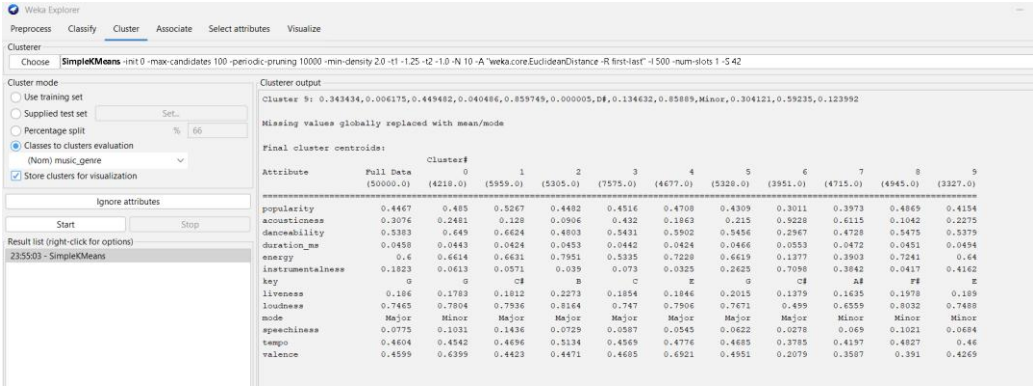
   - **numClusters**: Βάλε **10** (αφού ψάχνουμε 10 είδη).

   Προαιρετικό αλλά καλό: Άλλαξαμε το seed σε κάποιο άλλο νούμερο (π.χ. 42 ή 100) για να αλλάξει η αρχικοποίηση..

3. Στο "Cluster mode" (αριστερά), επιλεξαμε **Classes to clusters evaluation**. *Αυτό είναι το κλειδί:* Λέει στο Weka "Κρύψε το είδος μουσικής, φτιάξε ομάδες, και μετά πες μας αν πέτυχες τίποτα".
4. **Start**.



Εικόνα 24 Εφαρμογή SimpleKMeans με 10 clusters.



```
Cluster 9: 0.343434,0.006175,0.449482,0.040486,0.859749,0.000005,D#,0.134632,0.85885,Minor,0.304121,0.59235,0.123992

Missing values globally replaced with mean/mode

Final cluster centroids:
                                Cluster#
Attribute               Full Data          0          1          2          3          4          5          6          7          8          9
                        (50000.0)   (4218.0)   (5959.0)   (5305.0)   (7575.0)   (4677.0)   (5328.0)   (3951.0)   (4715.0)   (4945.0)   (3327.0)
==================================================================================================================================================
popularity                 0.4467     0.485     0.5267     0.4482     0.4516     0.4708     0.4309     0.3011     0.3973     0.4869     0.4154
acousticness               0.3076    0.2481      0.128     0.0906      0.432     0.1863      0.215     0.9228     0.6115     0.1042     0.2275
danceability               0.5383     0.649     0.6624     0.4803     0.5431     0.5902     0.5456     0.2967     0.4728     0.5475     0.5379
duration_ms                0.0458    0.0443     0.0424     0.0453     0.0442     0.0424     0.0466     0.0553     0.0472     0.0451     0.0494
energy                       0.6     0.6614     0.6631     0.7951     0.5335     0.7228     0.6619     0.1377     0.3903     0.7241       0.64
instrumentalness          0.1823     0.0613     0.0571      0.039     0.073     0.0325     0.2625     0.7098     0.3842     0.0417     0.4162
key                            G          G         C#          B          C          E          G         C#         A#         F#          E
liveness                   0.186     0.1783     0.1812     0.2273     0.1854     0.1846     0.2015     0.1379     0.1635     0.1978      0.189
loudness                  0.7465     0.7804     0.7936     0.8164      0.747     0.7906     0.7671      0.499     0.6559     0.8032     0.7488
mode                       Major      Minor      Major      Major      Major      Major      Major      Major      Minor      Minor      Minor
speechiness               0.0775     0.1031     0.1436     0.0729     0.0587     0.0545     0.0622     0.0278      0.069     0.1021     0.0684
tempo                     0.4604     0.4542     0.4696     0.5134     0.4569     0.4776     0.4685     0.3785     0.4197     0.4827       0.46
valence                   0.4599     0.6399     0.4423     0.4471     0.4685     0.6921     0.4951     0.2079     0.3587      0.391     0.4269
```

Εικόνα 25 Αποτελέσματα εφαρμογής SimpleKMeans με 10 clusters.

```
Clustered Instances

0       4218 (  8%)
1       5959 ( 12%)
2       5305 ( 11%)
3       7575 ( 15%)
4       4677 (  9%)
5       5328 ( 11%)
6       3951 (  8%)
7       4715 (  9%)
8       4945 ( 10%)
9       3327 (  7%)


Class attribute: music_genre
Classes to Clusters:

    0    1    2    3    4    5    6    7    8    9  <-- assigned to cluster
  377  650  550  423  227  819   79  426  709  740 | Electronic
  310  288  856  586  427  550  482  575  502  424 | Anime
  561  268  136  922  229  606  486  988  193  611 | Jazz
  392  529  966  778  434  533   24  279  799  266 | Alternative
  221  503  706 1305 1141  678   15   92  266   73 | Country
  677 1465  351  522  286  305    5  360  810  219 | Rap
  517  214  577 1247  717  770   93  273  320  272 | Blues
  377  474  801  972  857  579   41  165  511  223 | Rock
   62   30  119  288   52  181 2725 1185   93  265 | Classical
  724 1538  243  532  307  307    1  372  742  234 | Hip-Hop
```

Εικόνα 26 Confusion Matrix SimpleKMeans με 10 clusters.

```
Cluster  0 <-- Rap
Cluster  1 <-- Hip-Hop
Cluster  2 <-- Anime
Cluster  3 <-- Blues
Cluster  4 <-- Country
Cluster  5 <-- Rock
Cluster  6 <-- Classical
Cluster  7 <-- Jazz
Cluster  8 <-- Alternative
Cluster  9 <-- Electronic

Incorrectly clustered instances :       38710.0  77.42   %
```

Εικόνα 27 Αποτελέσματα SimpleKMeans με 10 clusters

Αποτελέσματα Συσταδοποίησης (Clustering - K-Means, k=10)

Εφαρμόσαμε τον αλγόριθμο SimpleKMeans ζητώντας τη δημιουργία 10 συστάδων (clusters), όσες και τα αρχικά είδη μουσικής.

1. **Διαχωρισιμότητα Classical:** Παρατηρήθηκε ότι το **Cluster 6** κατάφερε να απομονώσει με μεγάλη επιτυχία την Κλασική μουσική (2725 instances), εμφανίζοντας υψηλές τιμές στα χαρακτηριστικά Acousticness και Instrumentalness. Αυτό επιβεβαιώνει ότι η Κλασική μουσική έχει διακριτά δομικά χαρακτηριστικά.
2. **Σύγχυση Rap/Hip-Hop:** Το **Cluster 1** λειτούργησε ως "μαγνήτης" για τα Urban είδη, συγκεντρώνοντας σχεδόν ίσο αριθμό Rap (1465) και Hip-Hop (1538) εγγραφών. Αυτό

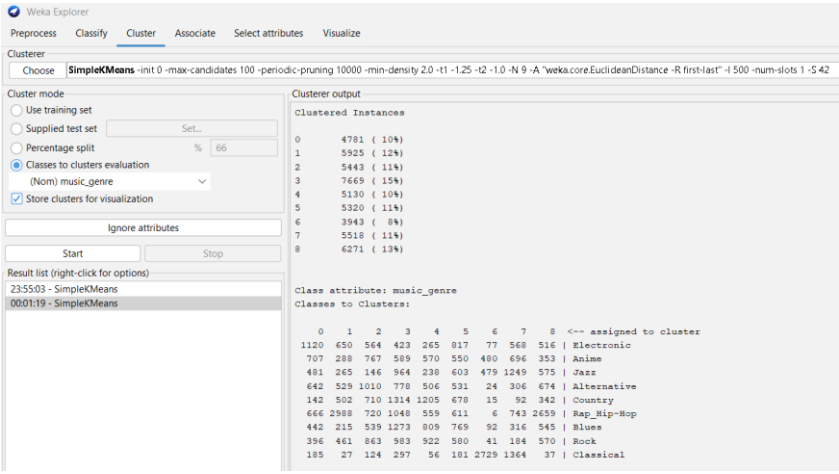
επιβεβαιώνει τα ευρήματα του Classification, ότι δηλαδή τα δύο είδη δεν διαχωρίζονται βάσει των συγκεκριμένων χαρακτηριστικών.

3. **Συνολική Απόδοση:** Το υψηλό ποσοστό λανθασμένης συσταδοποίησης (77.42%) υποδεικνύει ότι τα όρια μεταξύ των υπόλοιπων ειδών (Rock, Alternative, Country) είναι ασαφή στον χώρο των χαρακτηριστικών.

Περεταίρω προσπάθεια βελτιστοποίησης.

θα κάνουμε Clustering στο **Merged Dataset**. Περιμένουμε ότι αφού ενώσουμε Rap και Hip-Hop, το Clustering θα τα πάει λίγο καλύτερα (θα μειωθεί το λάθος), γιατί πλέον το Cluster 1 δεν θα θεωρείται "μισό λάθος".

1. **Preprocess**.
2. Ξεκλειδωσαμε το Class (No class).
3. **MergeTwoValues** (Rap + Hip-Hop -> Urban).
4. Κλειδωσαμε ξανά το Class (music_genre).
5. **Cluster**.
6. Βάλε **numClusters = 9** (ΠΡΟΣΟΧΗ: 9, όχι 10).
7. Τρέξαμε



Εικόνα 28 Αποτελέσματα εφαρμογής SimpleKMeans με 9 clusters

Βελτιστοποίηση Συσταδοποίησης (Merged Dataset - k=9)

Σε δεύτερο στάδιο, επαναλάβαμε τη διαδικασία συσταδοποίησης χρησιμοποιώντας το τροποποιημένο dataset (όπου Rap και Hip-Hop συγχωνεύθηκαν), ορίζοντας το πλήθος των συστάδων σε **k=9**.

Παρατηρήσεις:

1. **Μείωση Σφάλματος:** Το ποσοστό των λανθασμένα τοποθετημένων εγγραφών (Incorrectly clustered instances) μειώθηκε από **77.42%** σε **74.61%**. Η βελτίωση αυτή οφείλεται στην εξάλειψη της σύγχυσης μεταξύ των δύο Urban ειδών.
2. **Η Συστάδα της Classical:** Το **Cluster 6** παρέμεινε εξαιρετικά σταθερό, περιλαμβάνοντας σχεδόν αποκλειστικά κομμάτια Κλασικής μουσικής (2729

instances), επιβεβαιώνοντας ότι αποτελεί την πιο διακριτή κατηγορία του συνόλου δεδομένων.

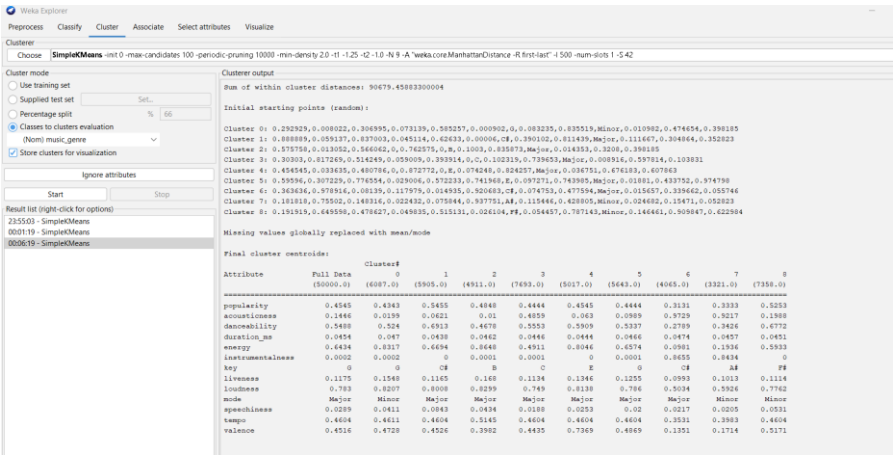
Η "Διασπορά" της νέας κατηγορίας: Λόγω του διπλάσιου μεγέθους της νέας κατηγορίας "Rap_Hip-Hop" (~10.000 εγγραφές), παρατηρούμε ότι κυριαρχεί στο **Cluster 1** (2988 εγγραφές), αλλά "ξεχειλίζει" και σε άλλα clusters (π.χ. στο Cluster 8), γεγονός που εξηγεί γιατί το σφάλμα παραμένει υψηλό.

Αλλαγή Απόστασης: Manhattan Distance

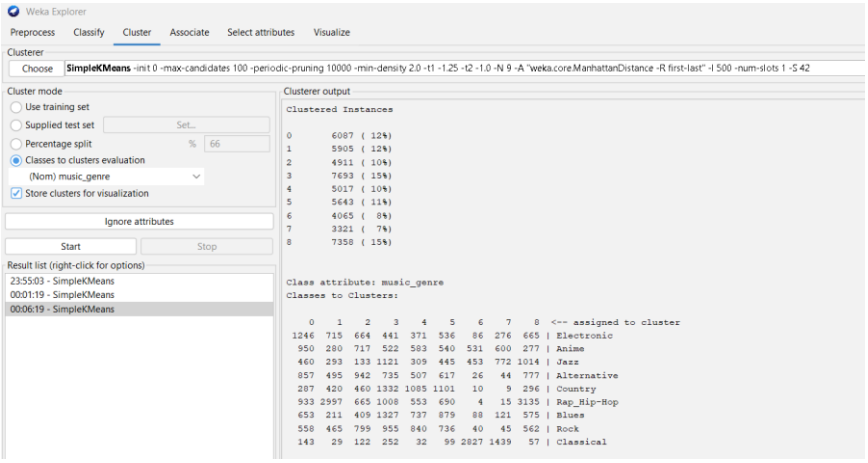
Ο K-Means από προεπιλογή χρησιμοποιεί την Ευκλείδεια απόσταση (ευθεία γραμμή). Σε δεδομένα με πολλές διαστάσεις μερικές φορές η απόσταση **Manhattan** δουλεύει καλύτερα γιατί είναι πιο ανθεκτική στον "θόρυβο".

Πώς το κάναμε:

1. Επέλεξαμε πάλι τον **SimpleKMeans**.
2. Κάναμε κλικ για ρυθμίσεις.
3. πεδίο **distanceFunction**.
4. το από EuclideanDistance σε **ManhattanDistance**.
5. **OK** και **Start**.



Εικόνα 29 Αποτελέσματα εφαρμογής SimpleKMeans με απόσταση Μανχάταν.



Εικόνα 30 Αποτελέσματα Confusion Matrix SimpleKMeans με απόσταση Μανχάταν.

Κατέβηκε από το 74.6% στο **74.2%**. Είναι οριακό, αλλά δείχνει ότι η αλλαγή μετρικής βοήθησε λίγο.

PCA (Principal Component Analysis)

PCA + K-Means

1. **Εφαρμογή PCA (Preprocess):**
    - καρτέλα **Preprocess**.
    - **Choose** -> weka > filters > unsupervised > attribute > **PrincipalComponents**.
    - **Apply**.

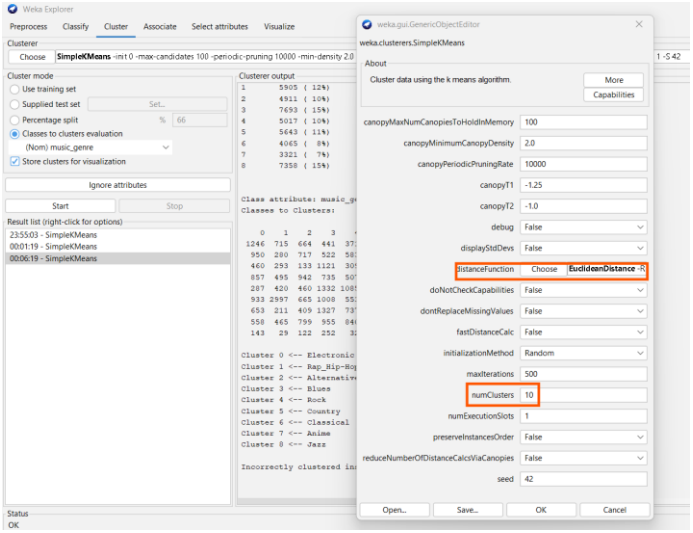
    Τα attributes να αλλάζουν όνομα. Αντί για popularity, tempo κλπ., βλέπουμε PC1, PC2, PC3... και πιθανώς λιγότερα σε πλήθος (το PCA κρατάει μόνο όσα έχουν σημαντική πληροφορία).

***Προσοχή**:* Το Class (music_genre) είναι κλειδωμένο (Target) ώστε να μην το πειράξει το PCA.

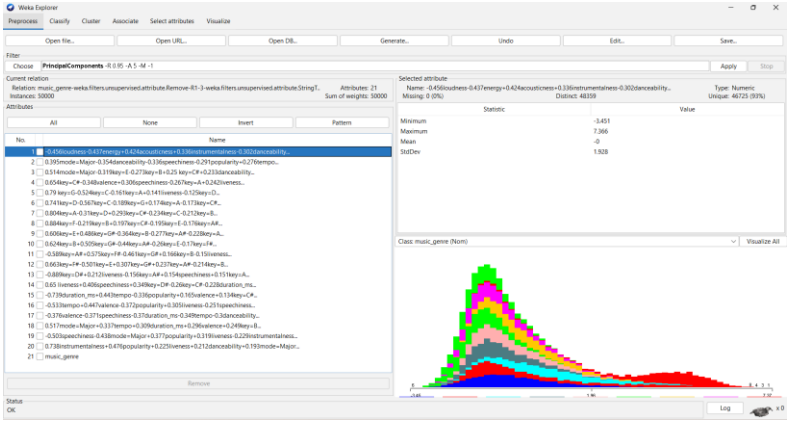
2. **Εκτέλεση Clustering (Cluster):**
    - **Cluster**.
    - Επιλεξαμε πάλι τον **SimpleKMeans**.

ΣΗΜΑΝΤΙΚΟ: Στις ρυθμίσεις το distanceFunction πίσω σε **EuclideanDistance** και βεβαιώσαμε ότι numClusters = **10**.

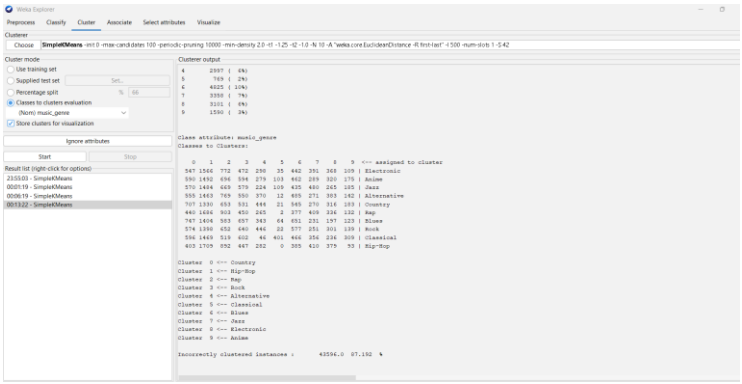
- Classes to clusters evaluation.
- **Start**.



Εικόνα 31 Εφαρμογή PCA + K-Means



Εικόνα 32 Αλλαγή στα features μετά την εφαρμογή PCA.



Εικόνα 33 Αποτελέσματα μετά την εφαρμογή PCA.

Αποτελέσματα

Το PCA **δεν βοήθησε καθόλου**. Το λάθος εκτοξεύθηκε στο **87.192%**. Το PCA συμπιέζει τα δεδομένα και σε datasets με τόσο μεγάλη επικάλυψη (overlap), χάνεται η λεπτομέρεια που χρειάζεται ο K-Means για να ξεχωρίσει τις "λεπτές γραμμές" μεταξύ των ειδών. Δοκιμάσαμε τεχνική μείωσης διαστάσεων (PCA) σε συνδυασμό με K-Means, ωστόσο η ακρίβεια επιδεινώθηκε (87.19% σφάλμα), υποδεικνύοντας ότι η απώλεια πληροφορίας από τη συμπίεση των δεδομένων επηρέασε αρνητικά τον διαχωρισμό των ήδη αλληλοεπικαλυπτόμενων κλάσεων. Η βέλτιστη επίδοση στο Clustering επιτεύχθηκε χρησιμοποιώντας **Manhattan Distance** στο Merged Dataset (74.21% σφάλμα)

Σύνοψη Αποτελεσμάτων Clustering K-Means

1. **K-Means (Euclidean):** 77.42% (Αρχικό)
2. **K-Means (Euclidean) + Merged Dataset:** 74.61%
3. **K-Means (Manhattan) + Merged Dataset: 74.21%** (ΝΙΚΗΤΗΣ)
4. **PCA + K-Means:** 87.19% (Αποτυχία)

4.2. Ιεραρχική Συσταδοποίηση

Στη συνέχεια, εξετάστηκε η Ιεραρχική Συσταδοποίηση.

- **Τεχνικός Περιορισμός:** Ο αλγόριθμος HierarchicalClusterer έχει εξαιρετικά υψηλές απαιτήσεις σε μνήμη και υπολογιστική ισχύ, καθώς προσπαθεί να συγκρίνει κάθε τραγούδι

με όλα τα υπόλοιπα. Για ένα dataset 50.000 εγγραφών, η εκτέλεση ήταν πρακτικά αδύνατη με τους διαθέσιμους πόρους.

• **Λύση:** Εφαρμόστηκε φίλτρο δειγματοληψίας (Resample) στο 1% των δεδομένων, μειώνοντας το σύνολο σε 500 αντιπροσωπευτικές εγγραφές, ώστε να καταστεί εφικτή η ανάλυση.

• **Αποτελέσματα:** Incorrectly clustered instances: 83.6 %

```
=== Model and evaluation on training set ===

Clustered Instances

0       173 ( 35%)
1       319 ( 64%)
2         1 (  0%)
3         1 (  0%)
4         1 (  0%)
5         1 (  0%)
6         1 (  0%)
7         1 (  0%)
8         1 (  0%)
9         1 (  0%)
```

```
Class attribute: music_genre
Classes to Clusters:

  0  1  2  3  4  5  6  7  8  9  <-- assigned to cluster
 17 25  0  0  1  0  0  0  0  0 | Electronic
 18 17  0  0  0  0  0  0  0  0 | Anime
 18 19  0  0  0  1  0  1  0  0 | Jazz
 20 26  0  0  0  0  0  0  1  0 | Alternative
  7 52  0  0  0  0  0  0  0  0 | Country
 17 29  0  1  0  0  0  0  0  0 | Rap
 18 37  0  0  0  0  1  0  0  0 | Blues
 16 41  0  0  0  0  0  0  0  1 | Rock
 19 42  1  0  0  0  0  0  0  0 | Classical
 23 31  0  0  0  0  0  0  0  0 | Hip-Hop
```

```
Cluster  0 <-- Hip-Hop
Cluster  1 <-- Country
Cluster  2 <-- Classical
Cluster  3 <-- Rap
Cluster  4 <-- Electronic
Cluster  5 <-- No class
Cluster  6 <-- Blues
Cluster  7 <-- Jazz
Cluster  8 <-- Alternative
Cluster  9 <-- Rock

Incorrectly clustered instances :       418.0    83.6    %
```

Εικόνα 34 Αποτελέσματα HierarchicalClusterer.

4.3. Συσταδοποίηση Βάσει Πυκνότητας

Τέλος, εφαρμόστηκε ο πιθανοτικός αλγόριθμος EM, ο οποίος αναθέτει εγγραφές σε συστάδες βάσει της πιθανότητας να ανήκουν σε μια συγκεκριμένη κατανομή. Πλήθος συστάδων N=10.

Incorrectly clustered instances: 75.80 %

```
=== Model and evaluation on training set ===

Clustered Instances

0       1728 (  3%)
1       3871 (  8%)
2      16260 ( 33%)
3       9131 ( 18%)
4       6861 ( 14%)
5       1711 (  3%)
6       2160 (  4%)
7       3389 (  7%)
8       3388 (  7%)
9       1506 (  3%)

Log likelihood: 12.92414
```

```
Cluster  0 <-- Rap
Cluster  1 <-- Electronic
Cluster  2 <-- Country
Cluster  3 <-- Hip-Hop
Cluster  4 <-- Blues
Cluster  5 <-- Anime
Cluster  6 <-- Rock
Cluster  7 <-- Jazz
Cluster  8 <-- Classical
Cluster  9 <-- Alternative

Incorrectly clustered instances :       37903.0  75.7984 %
```

```
Class attribute: music_genre
Classes to Clusters:

    0    1    2    3    4    5    6    7    8    9  <-- assigned to cluster
   50 1365  987  794  120  164   98 1174   55  193 | Electronic
  338  502 1011  392  560  590  195  438  447  527 | Anime
  244  725  387  441 1072   41  608 1005  415   62 | Jazz
    6  304 2111 1264  483  245  178  185   22  202 | Alternative
    0   29 2932  527 1329   66   65    8    6   38 | Country
    1   40 2538 2089  161   18   98   30    3   22 | Rap
   13  470 1173  565 1470  363  355  251   86  254 | Blues
    8  275 2483  916  750  155  112  133   33  135 | Rock
 1068  107   78   44  761   64  371  122 2321   64 | Classical
    0   54 2560 2099  150    5   80   43    0    9 | Hip-Hop
```

Εικόνα 35 Αποτελέσματα αλγόριθμου EM

4.4. Συγκριτική Αξιολόγηση και Συμπεράσματα Συσταδοποίησης

Η συγκριτική μελέτη των τριών αλγορίθμων οδηγεί σε σημαντικά συμπεράσματα για τη φύση των μουσικών δεδομένων:

1. Σύγκριση Επιδόσεων. Ο αλγόριθμος K-Means πέτυχε το χαμηλότερο ποσοστό σφάλματος ξεπερνώντας τόσο τον EM (75.8%) όσο και τον Ιεραρχικό (83.6%). Παρότι τα ποσοστά λάθους είναι υψηλά σε απόλυτες τιμές, ο K-Means κατάφερε να δημιουργήσει την πιο ισορροπημένη κατανομή συστάδων, αποφεύγοντας τη δημιουργία ακραία άνισων ομάδων.

2. Αποτυχία της Ιεραρχικής Μεθόδου. Ο Ιεραρχικός αλγόριθμος απέτυχε να εντοπίσει δομή στα δεδομένα. Εμφάνισε το φαινόμενο της "αλυσίδας" (chaining effect), δημιουργώντας δύο τεράστιες "υπερ-συστάδες" που περιείχαν το 99% των τραγουδιών και αφήνοντας τις υπόλοιπες σχεδόν κενές.

3. Κοινά Ευρήματα. Και οι τρεις αλγόριθμοι συμφώνησαν σε ένα σημείο: Η Κλασική μουσική (Classical) διαθέτει τόσο διακριτά ακουστικά χαρακτηριστικά (χαμηλή ένταση, υψηλή ακουστικότητα), που απομονώθηκε επιτυχώς σε ξεχωριστή συστάδα από όλες τις μεθόδους. Αντίθετα, είδη όπως Rock, Pop και Alternative παρουσίασαν μεγάλη αλληλοεπικάλυψη στον χώρο των χαρακτηριστικών, με αποτέλεσμα να ομαδοποιούνται μαζικά σε κοινές συστάδες (όπως φάνηκε έντονα στο Cluster 2 του EM).

5. Τελικά Συμπεράσματα Εργασίας

Η παρούσα εργασία μελέτησε διεξοδικά το πρόβλημα της ανάλυσης μουσικών δεδομένων από το Spotify. Μέσα από τα στάδια της προ-επεξεργασίας, της κατηγοριοποίησης και της συσταδοποίησης, προέκυψαν τα εξής συνολικά ευρήματα:

1. Η Πολυπλοκότητα του Ήχου: Η μουσική δεν μπαίνει εύκολα σε "κουτιά". Είδη όπως το Rap και το Hip-Hop είναι μαθηματικά σχεδόν πανομοιότυπα, καθιστώντας τον αυτόματο διαχωρισμό τους πρόκληση ακόμα και για εξελιγμένους αλγορίθμους.

2. Η Αξία της Μηχανικής Μάθησης: Παρά τις δυσκολίες, ο αλγόριθμος Random Forest κατάφερε να "ξεκλειδώσει" κρυμμένα μοτίβα, πετυχαίνοντας την καλύτερη απόδοση.

3. Ο Ρόλος των Δεδομένων: Η στρατηγική μας να διατηρήσουμε τα outliers και να διορθώσουμε τα ελλιπή δεδομένα (αντί να τα διαγράψουμε) αποδείχθηκε καθοριστική, επιτρέποντας στα μοντέλα να εκπαιδευτούν σε ένα πλούσιο και ρεαλιστικό σύνολο δεδομένων. Η ανάλυση κατέδειξε ότι η Εξόρυξη Γνώσης μπορεί να προσφέρει πολύτιμες πληροφορίες για τη δομή της μουσικής, ανοίγοντας τον δρόμο για πιο έξυπνα συστήματα μουσικών προτάσεων (recommender systems).